

Deep Reinforcement Learning

Kian Katanforoosh

Today's outline

I. Motivation

II. Recycling is good: an introduction to RL

III. Deep Q-Learning

IV. Application of Deep Q-Learning: Breakout (Atari)

V. Tips to train Deep Q-Network

VI. Advanced topics

I. Motivation



Human Level Control
through Deep
Reinforcement Learning

Mastering the Game of Go without Human Knowledge

David Silver*, Julian Schrittwieser*, Karen Simonyan*, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, Yutian Chen, Timothy Lillicrap, Fan Hui, Laurent Sifre, George van den Driessche, Thore Graepel, Demis Hassabis.

DeepMind, 5 New Street Square, London EC4A 3TW.

*These authors contributed equally to this work.

A long-standing goal of artificial intelligence is an algorithm that learns, *tabula rasa*, superhuman proficiency in challenging domains. Recently, *AlphaGo* became the first program to defeat a world champion in the game of Go. The tree search in *AlphaGo* evaluated positions and selected moves using deep neural networks. These neural networks were trained by supervised learning from human expert moves, and by reinforcement learning from self-play. Here, we introduce an algorithm based solely on reinforcement learning, without human data, guidance, or domain knowledge beyond game rules. *AlphaGo* becomes its own teacher: a neural network is trained to predict *AlphaGo*'s own move selections and also the winner of *AlphaGo*'s games. This neural network improves the strength of tree search, resulting in higher quality move selection and stronger self-play in the next iteration. Starting *tabula rasa*, our new program *AlphaGo Zero* achieved superhuman performance, winning 100-0 against the previously published, champion-defeating *AlphaGo*.

AlphaGo



AlphaStar

Training language models to follow instructions with human feedback

Long Ouyang* Jeff Wu* Xu Jiang* Diogo Almeida* Carroll L. Wainwright*
Pamela Mishkin* Chong Zhang Sandhini Agarwal Katarina Slama Alex Ray
John Schulman Jacob Hilton Fraser Kelton Luke Miller Maddie Simens
Amanda Askell¹ Peter Welinder Paul Christiano*[†]
Jan Leike* Ryan Lowe*
OpenAI

Abstract

Making language models bigger does not inherently make them better at following a user's intent. For example, large language models can generate outputs that are untruthful, toxic, or simply not helpful to the user. In other words, these models are not *aligned* with their users. In this paper, we show an avenue for aligning language models with user intent on a wide range of tasks by fine-tuning with human feedback. Starting with a set of labeler-written prompts and prompts submitted through the OpenAI API, we collect a dataset of labeler demonstrations of the desired model behavior, which we use to fine-tune GPT-3 using supervised learning. We then collect a dataset of rankings of model outputs, which we use to further fine-tune this supervised model using reinforcement learning from human feedback. We call the resulting models *InstructGPT*. In human evaluations on our prompt distribution, outputs from the 1.3B parameter InstructGPT model are preferred to outputs from the 175B GPT-3, despite having 100x fewer parameters. Moreover, InstructGPT models show improvements in truthfulness and reductions in toxic output generation while having minimal performance regressions on public NLP datasets. Even though InstructGPT still makes simple mistakes, our results show that fine-tuning with human feedback is a promising direction for aligning language models with human intent.

1 Introduction

Large language models (LMs) can be "prompted" to perform a range of natural language processing (NLP) tasks, given some examples of the task as input. However, these models often express unintended behaviors such as making up facts, generating biased or toxic text, or simply not following user instructions (Bender et al., 2021; Bommasani et al., 2021; Kenton et al., 2021; Weidinger et al., 2021; Tamkin et al., 2021; Gehrmann et al., 2020). This is because the language modeling objective

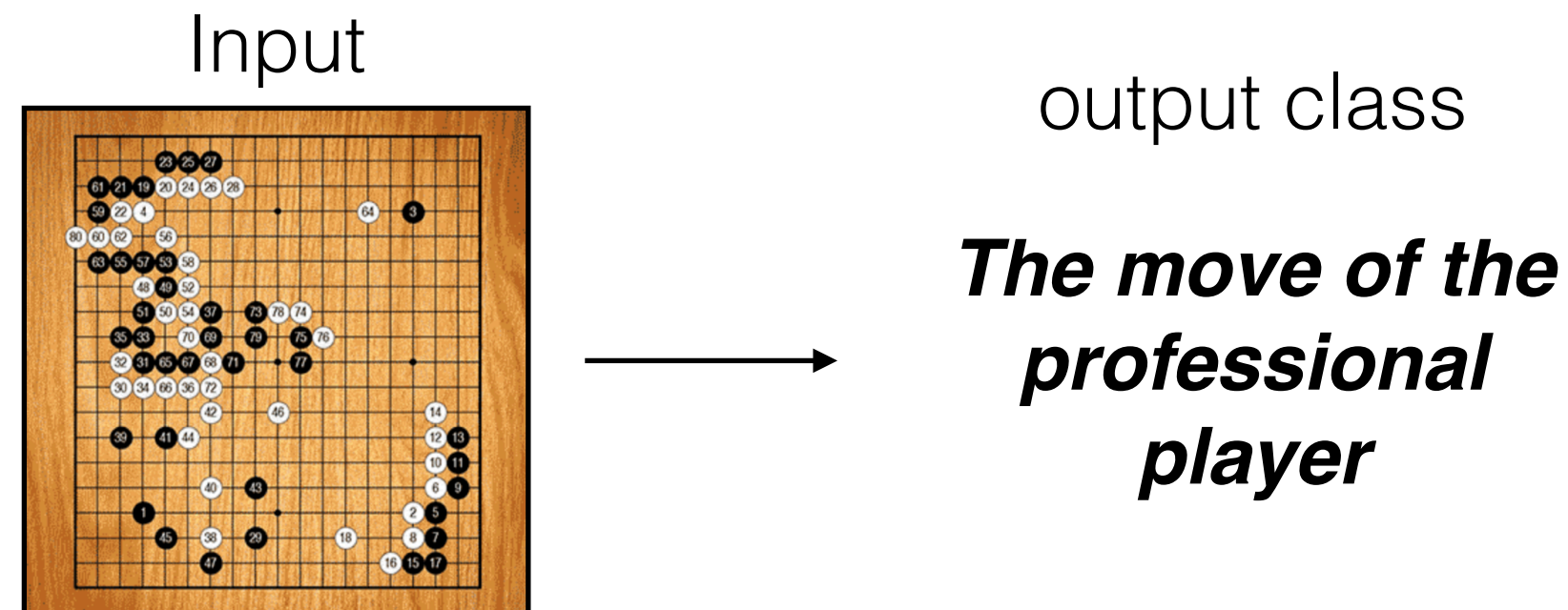
^{*}Primary authors. This was a joint project of the OpenAI Alignment team. RL and JL are the team leads.
Corresponding author: lowe@openai.com.
[†]Work done while at OpenAI. Current affiliations: AA: Anthropic; PC: Alignment Research Center.

Reinforcement Learning with
Human Feedback

[Ouyang, Wu, Jiang, Almeida, Wainwright, Mishkin, et al. (2022): Training language models to follow instructions with human feedback]
[Vinyals et al. (2019): Grandmaster level in StarCraft II using multi-agent reinforcement learning]
[Silver, Schrittwieser, Simonyan et al. (2017): Mastering the game of Go without human knowledge]
[Mnih, Kavukcuoglu, Silver et al. (2015): Human Level Control through Deep Reinforcement Learning]

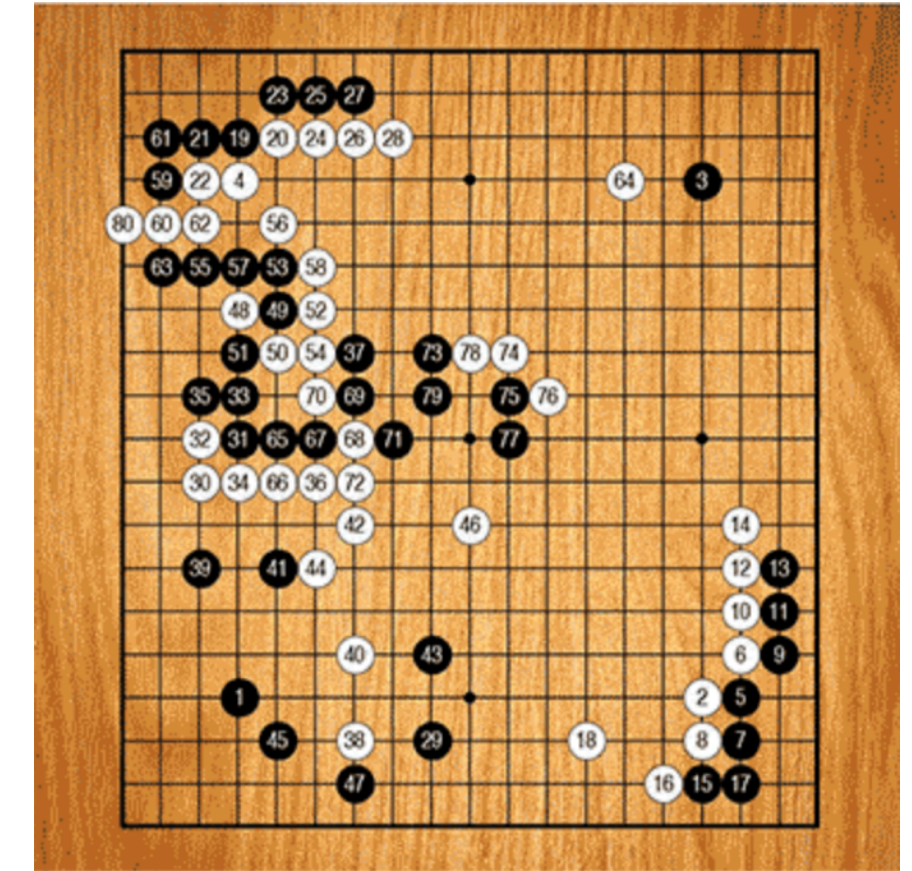
I. Motivation

How would you solve Go with classic supervised learning?



issues:

- Ground truth probably wrongly defined.
- Too many states in this Game.
- We will likely not generalize.



Source: <https://deepmind.com/blog/alphago-zero-learning-scratch/>

Why RL?

- Delayed labels
- Making sequences of decisions

What is RL?

- Automatically learn to make good sequences of decision
- Teaching by experience vs. Teaching by example.

Examples of RL applications

Games



Robotics

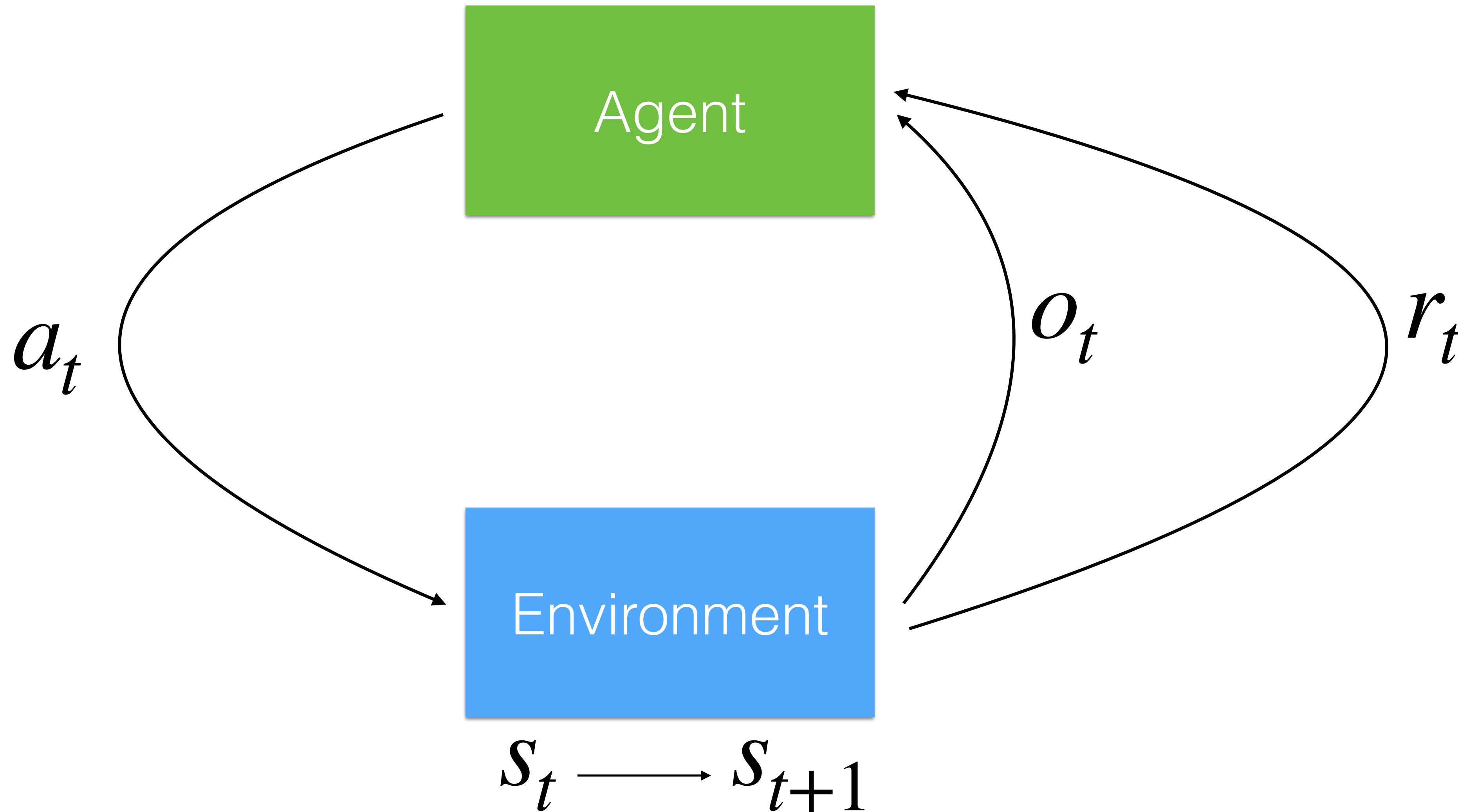


Advertisement



I. Motivation

Transition: $s_t \rightarrow a_t \rightarrow (o_t, r_t) \rightarrow s_{t+1}$



Today's outline

I. Motivation

II. Recycling is good: an introduction to RL

III. Deep Q-Networks

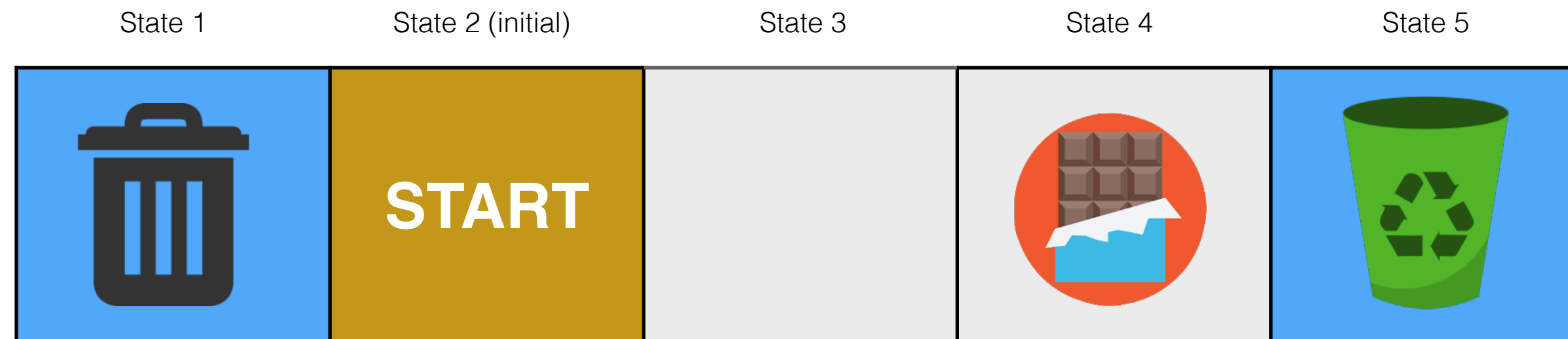
IV. Application of Deep Q-Network: Breakout (Atari)

V. Tips to train Deep Q-Network

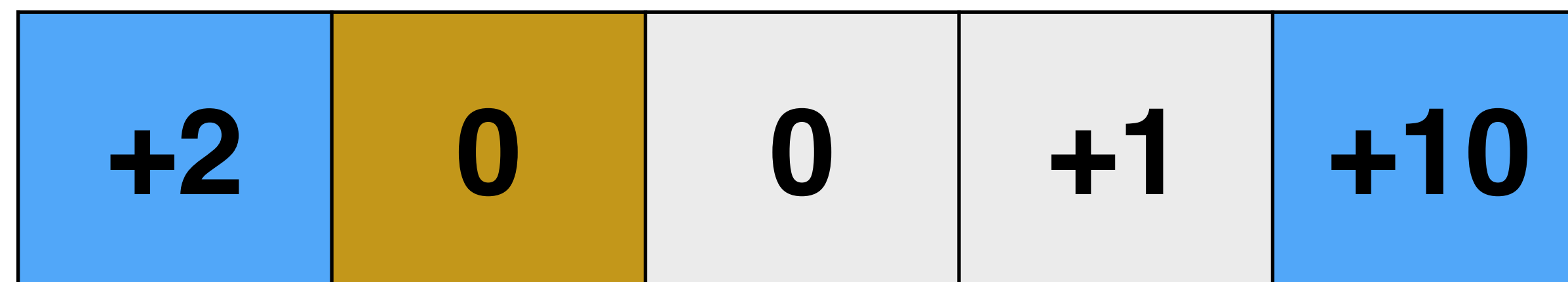
VI. Advanced topics

II. Recycling is good: an introduction to RL

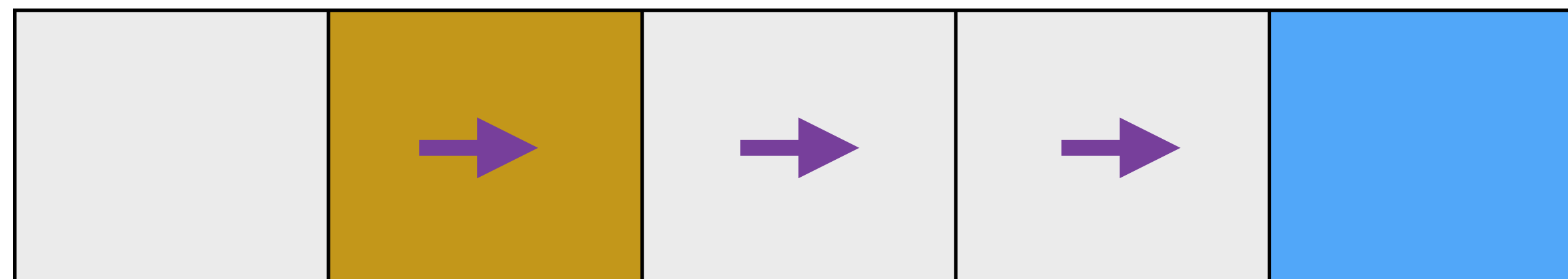
Problem statement



Define reward “ r ” in every state



Best strategy to follow if $\gamma = 1$



Goal: maximize the return (rewards)

Number of states: 5

Types of states:

Agent's Possible actions:

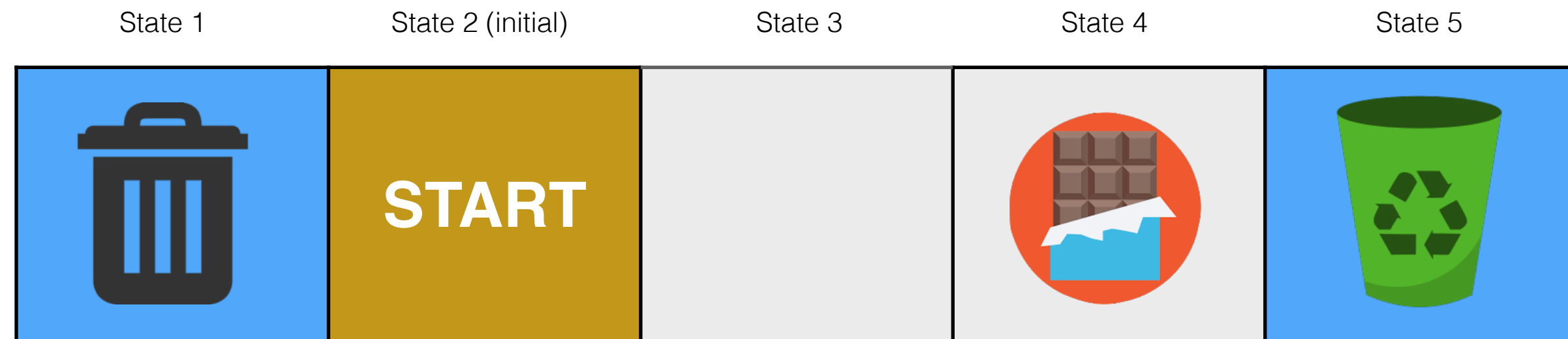
Additional rule: garbage collector coming in 3min, it takes 1min to move between states

How to define the long-term return?

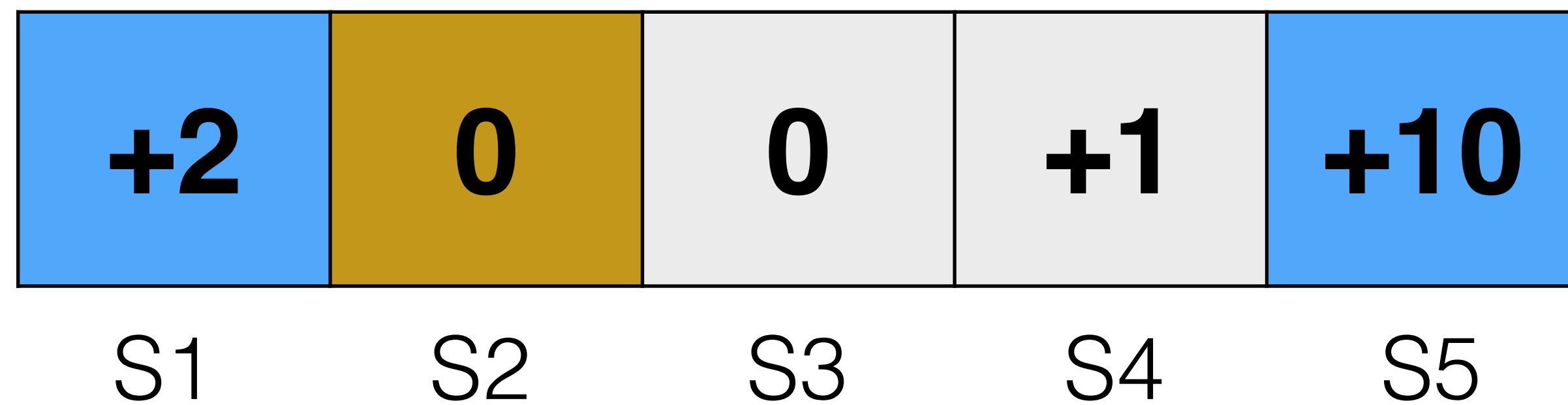
Discounted return $R = \sum_{t=0}^{\infty} \gamma^t r_t = r_0 + \gamma r_1 + \gamma^2 r_2 + \dots$

II. Recycling is good: an introduction to RL

Problem statement



Define reward “**r**” in every state



Assuming $\gamma = 0.9$

Discounted return $R = \sum_{t=0} \gamma^t r_t = r_0 + \gamma r_1 + \gamma^2 r_2 + \dots$

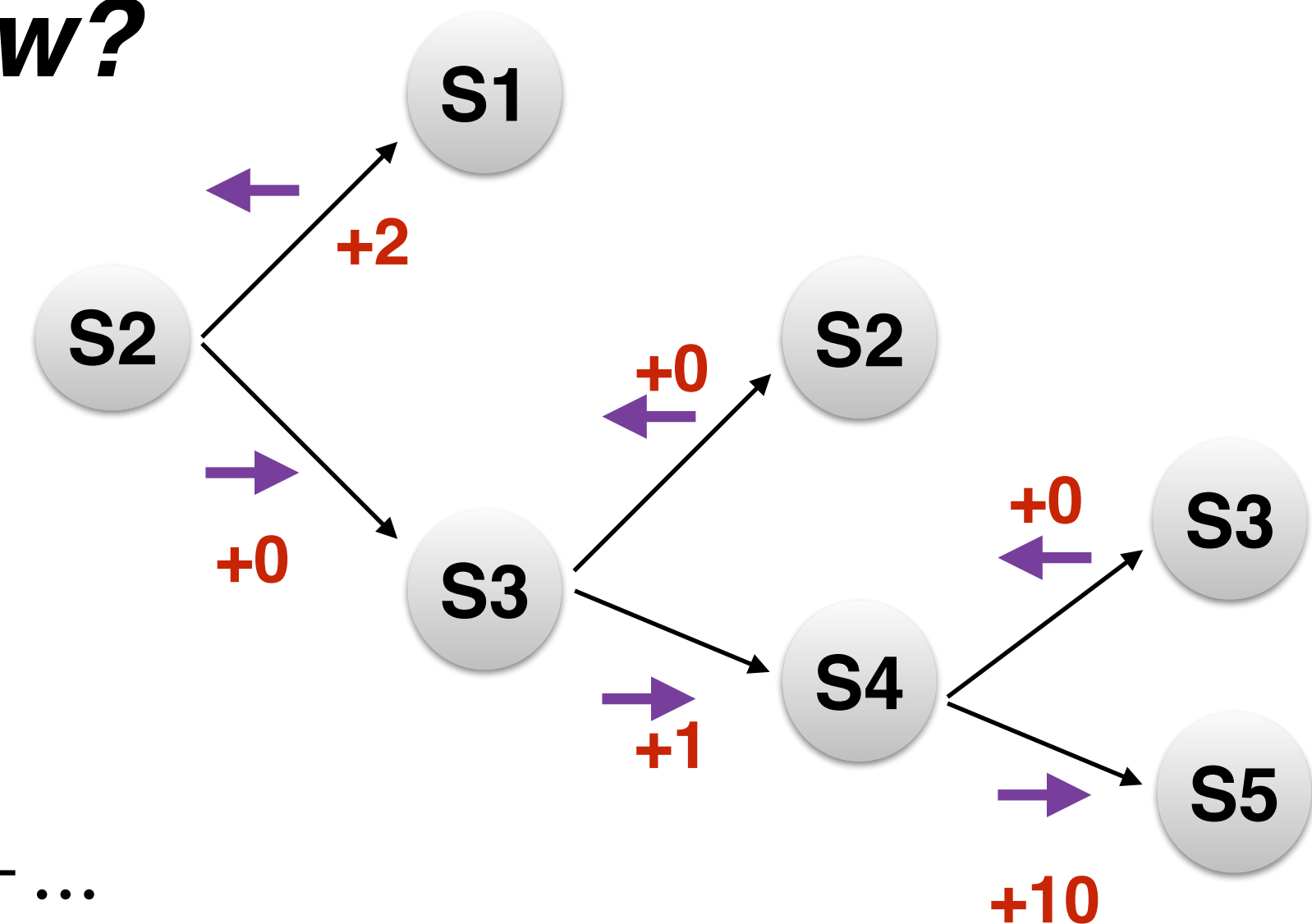
What do we want to learn?

how good is it to take
action 1 in state 2

Q-table

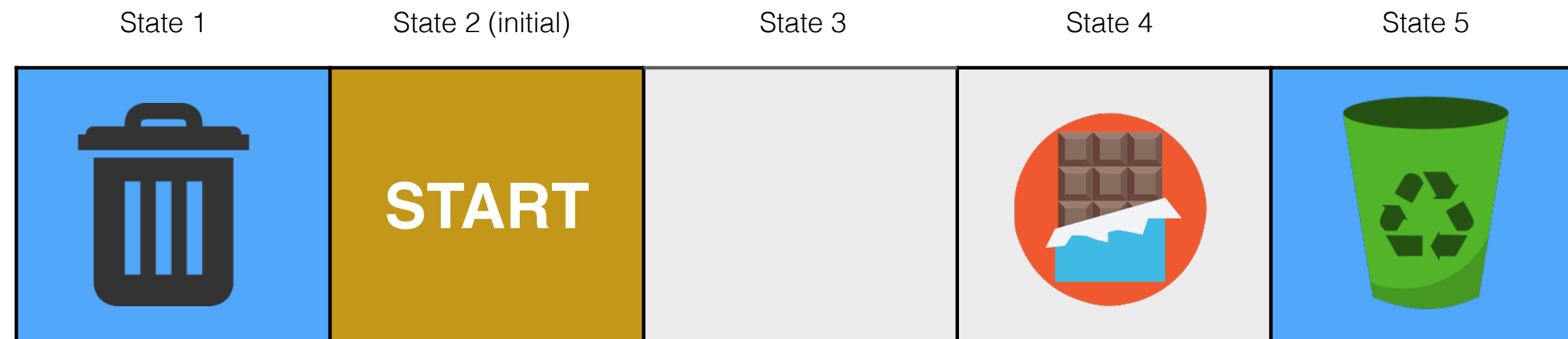
$$Q = \begin{matrix} & \xrightarrow{\text{\#actions}} \\ \begin{matrix} \downarrow \text{\#states} \\ \begin{pmatrix} Q_{11} & Q_{12} \\ Q_{21} & Q_{22} \\ Q_{31} & Q_{32} \\ Q_{41} & Q_{42} \\ Q_{51} & Q_{52} \end{pmatrix} \end{matrix} \end{matrix}$$

How?

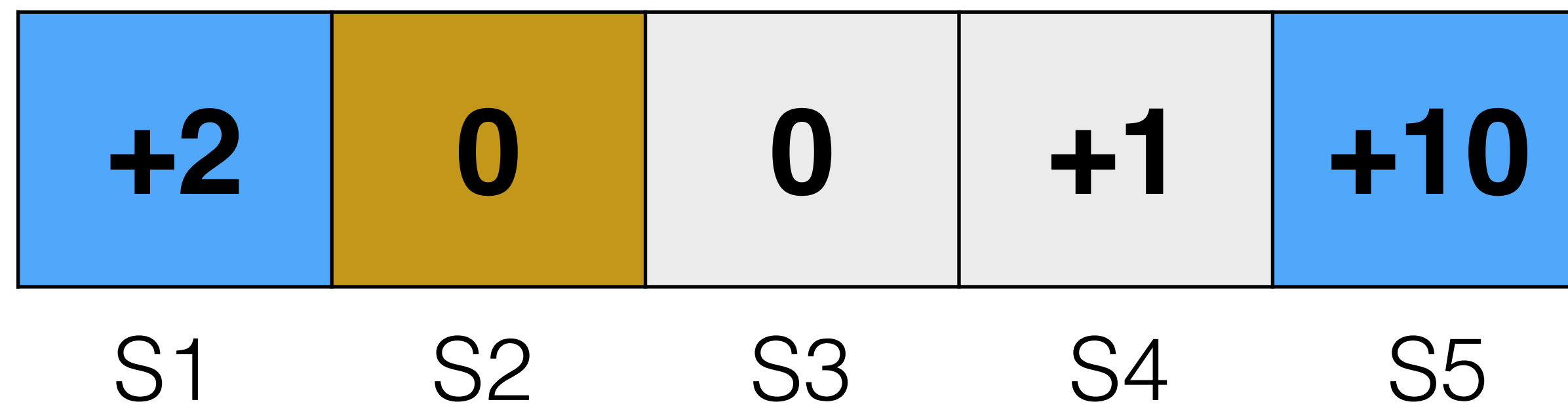


II. Recycling is good: an introduction to RL

Problem statement



Define reward “**r**” in every state



Assuming $\gamma = 0.9$

Discounted return $R = \sum_{t=0} \gamma^t r_t = r_0 + \gamma r_1 + \gamma^2 r_2 + \dots$

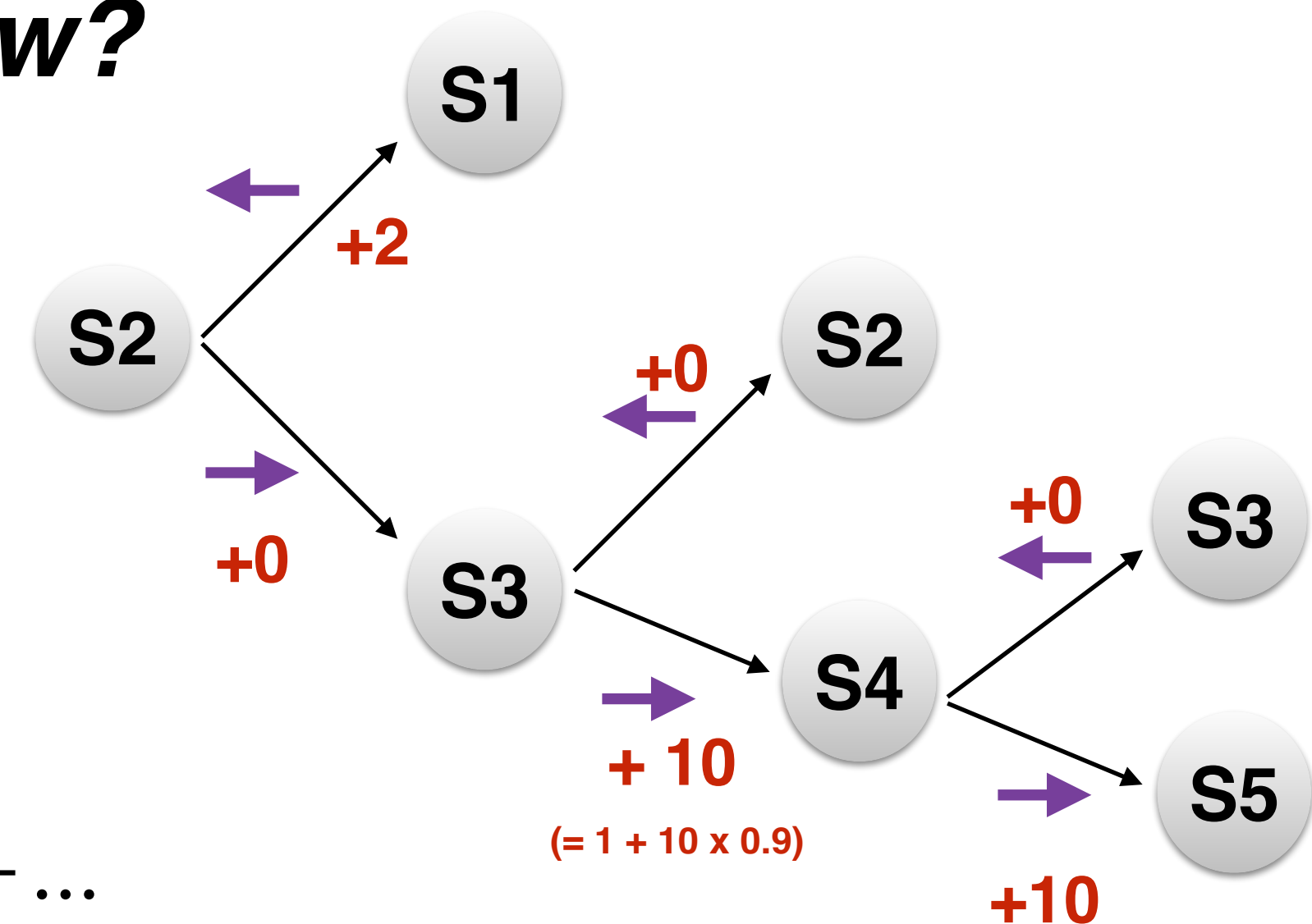
What do we want to learn?

how good is it to take
action 1 in state 2

Q-table

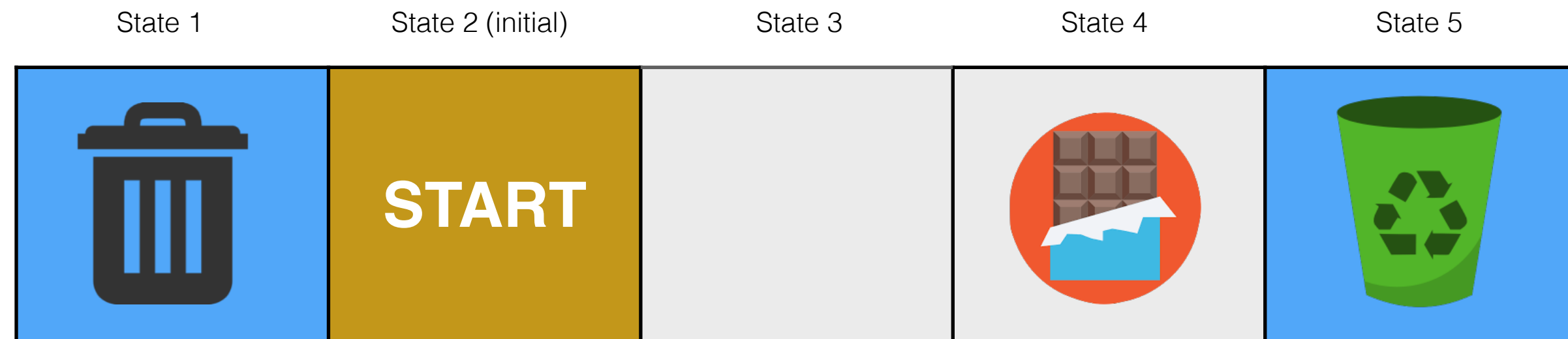
$$Q = \begin{matrix} & \xrightarrow{\text{\#actions}} \\ \begin{matrix} \downarrow \text{\#states} \\ \begin{pmatrix} Q_{11} & Q_{12} \\ Q_{21} & Q_{22} \\ Q_{31} & Q_{32} \\ Q_{41} & Q_{42} \\ Q_{51} & Q_{52} \end{pmatrix} \end{matrix} \end{matrix}$$

How?

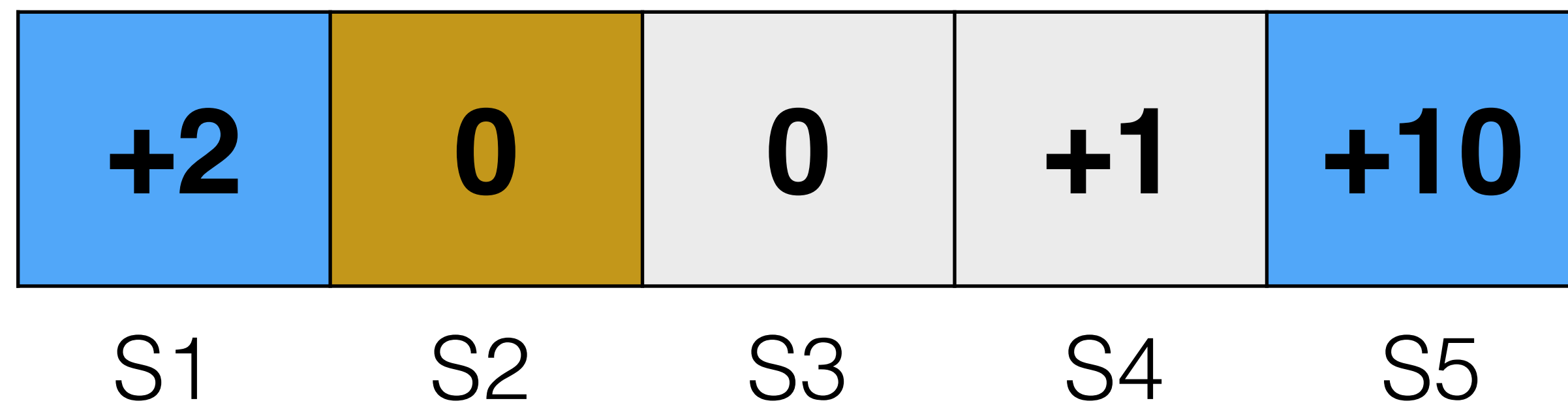


II. Recycling is good: an introduction to RL

Problem statement



Define reward “**r**” in every state



Assuming $\gamma = 0.9$

Discounted return $R = \sum_{t=0} \gamma^t r_t = r_0 + \gamma r_1 + \gamma^2 r_2 + \dots$

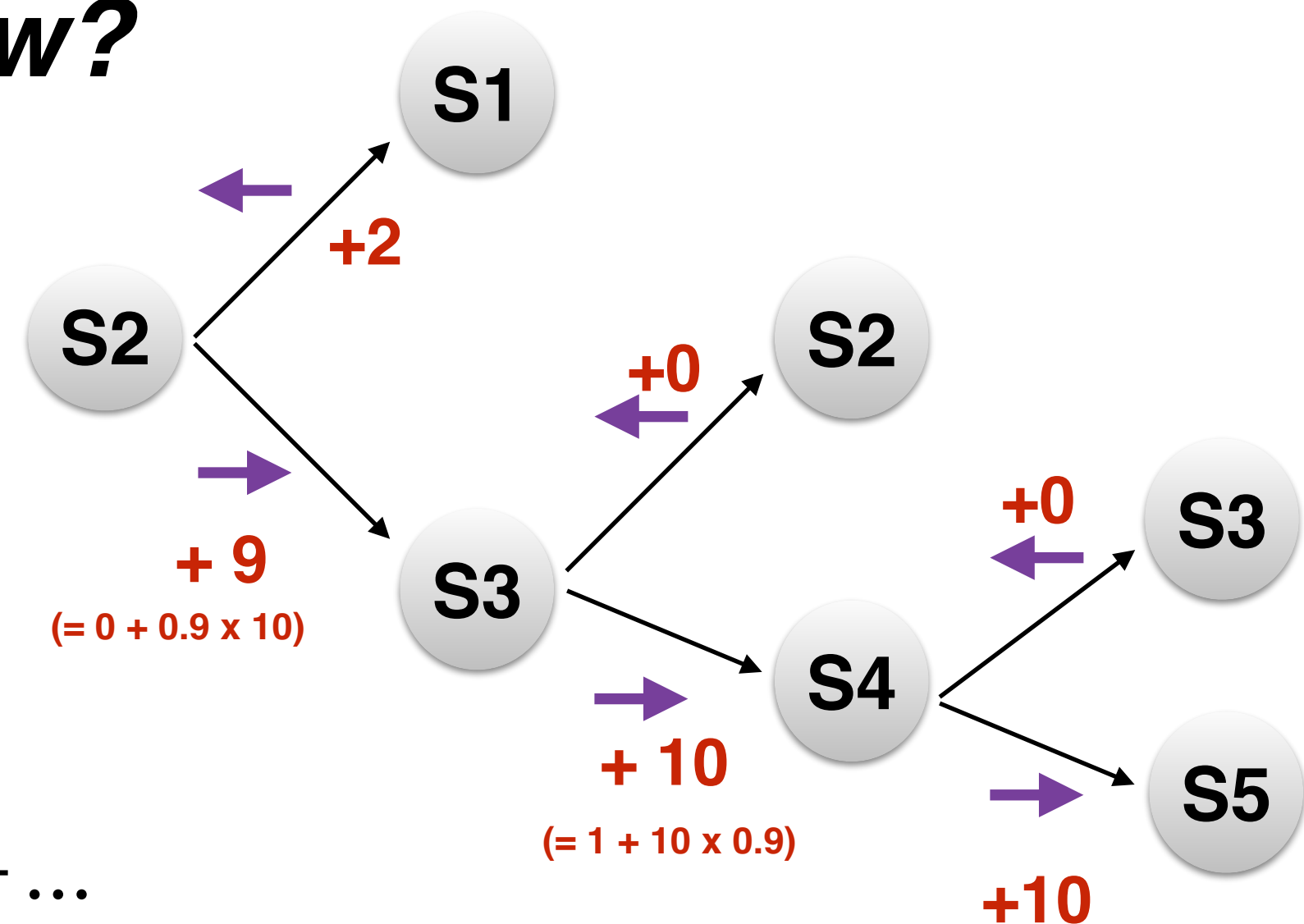
What do we want to learn?

how good is it to take
action 1 in state 2

Q-table

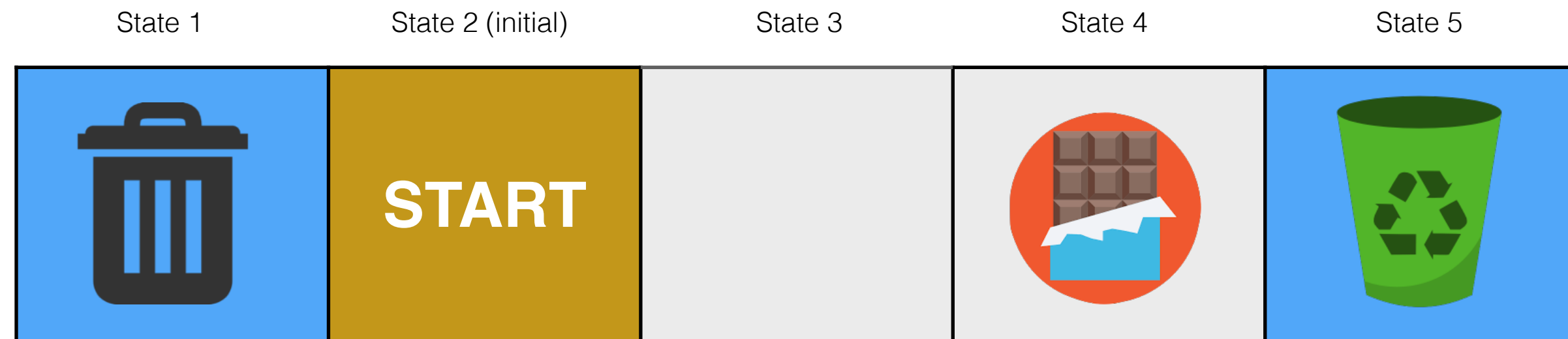
$$Q = \begin{matrix} & \xrightarrow{\text{\#actions}} \\ \begin{matrix} \downarrow \text{\#states} \\ \begin{pmatrix} Q_{11} & Q_{12} \\ Q_{21} & Q_{22} \\ Q_{31} & Q_{32} \\ Q_{41} & Q_{42} \\ Q_{51} & Q_{52} \end{pmatrix} \end{matrix} \end{matrix}$$

How?

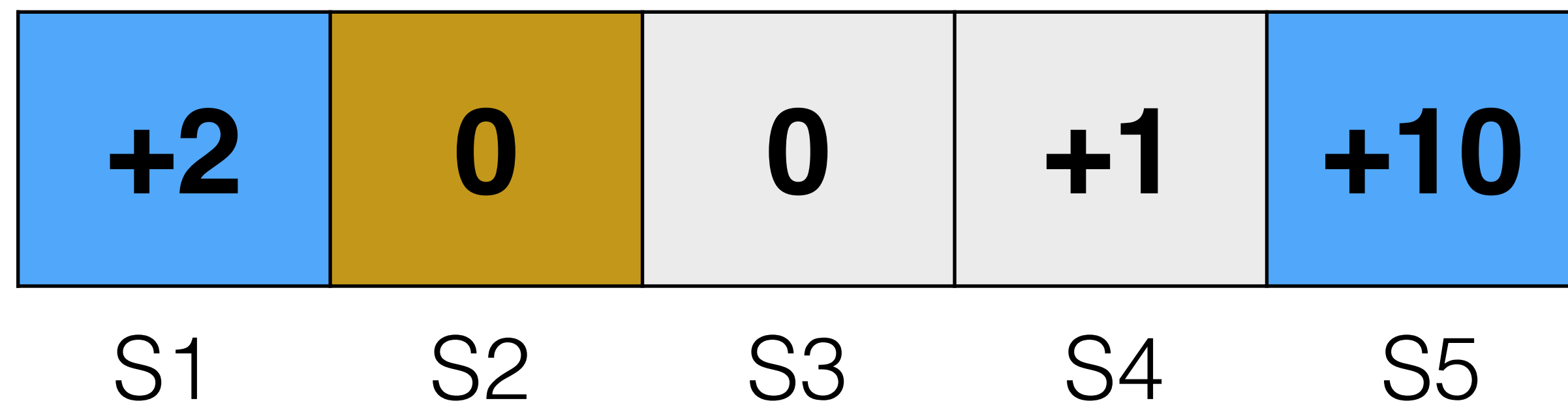


II. Recycling is good: an introduction to RL

Problem statement



Define reward “ r ” in every state



Assuming $\gamma = 0.9$

Discounted return $R = \sum_{t=0} \gamma^t r_t = r_0 + \gamma r_1 + \gamma^2 r_2 + \dots$

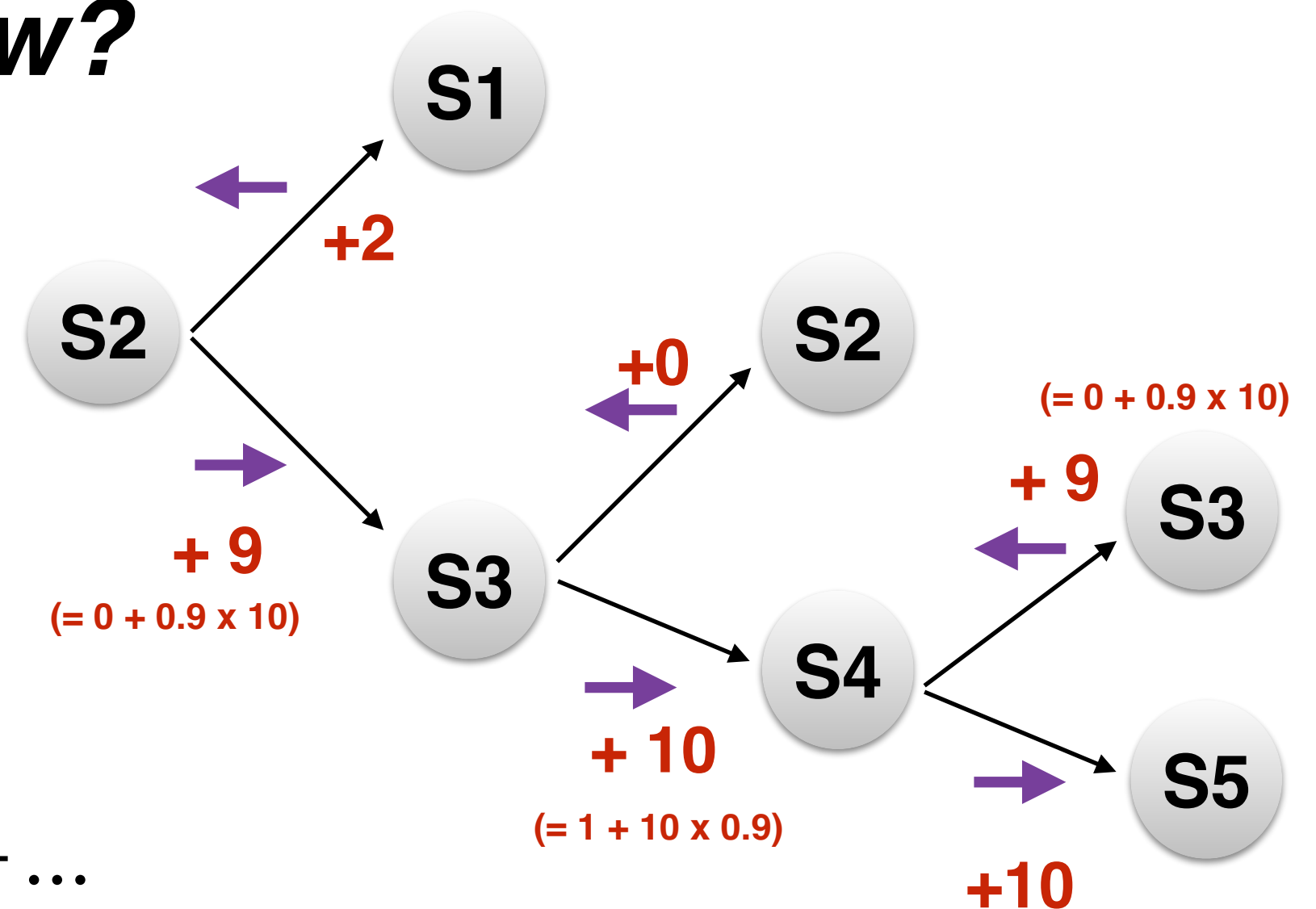
What do we want to learn?

how good is it to take action 1 in state 2

Q -table

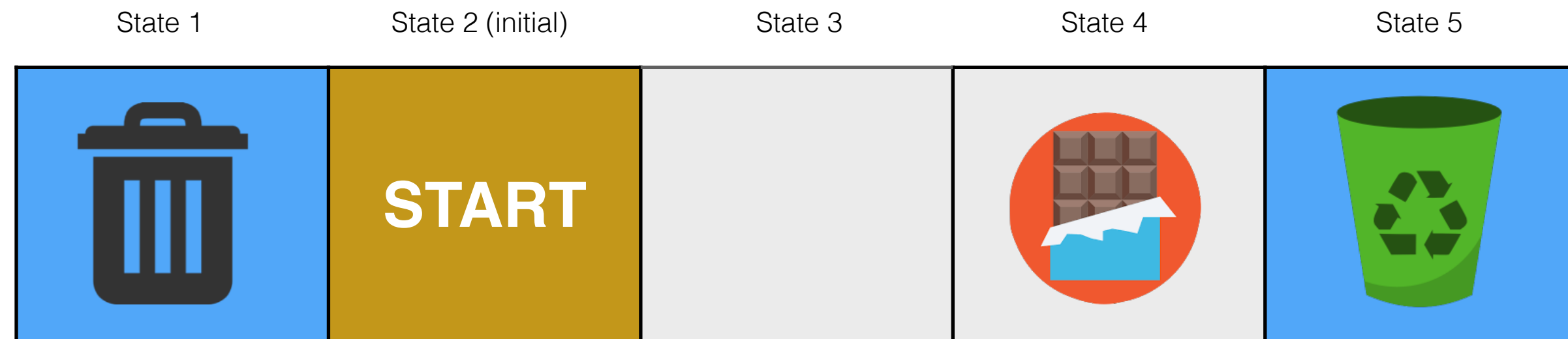
$$Q = \begin{matrix} & \xrightarrow{\text{\#actions}} \\ \begin{matrix} \downarrow \text{\#states} \\ \begin{pmatrix} Q_{11} & Q_{12} \\ Q_{21} & Q_{22} \\ Q_{31} & Q_{32} \\ Q_{41} & Q_{42} \\ Q_{51} & Q_{52} \end{pmatrix} \end{matrix} \end{matrix}$$

How?

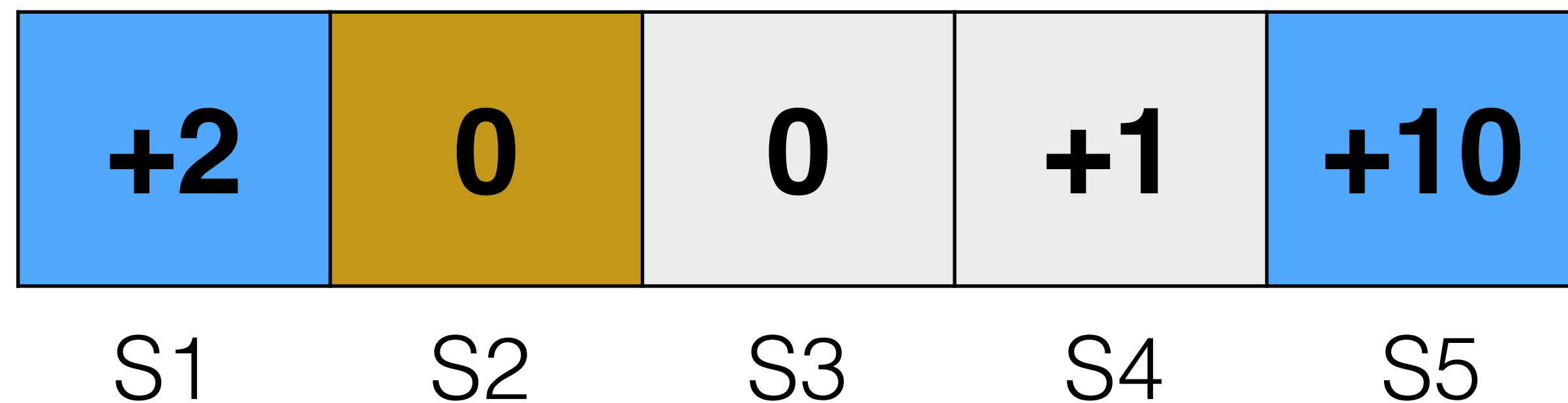


II. Recycling is good: an introduction to RL

Problem statement



Define reward “ r ” in every state



Assuming $\gamma = 0.9$

Discounted return $R = \sum_{t=0} \gamma^t r_t = r_0 + \gamma r_1 + \gamma^2 r_2 + \dots$

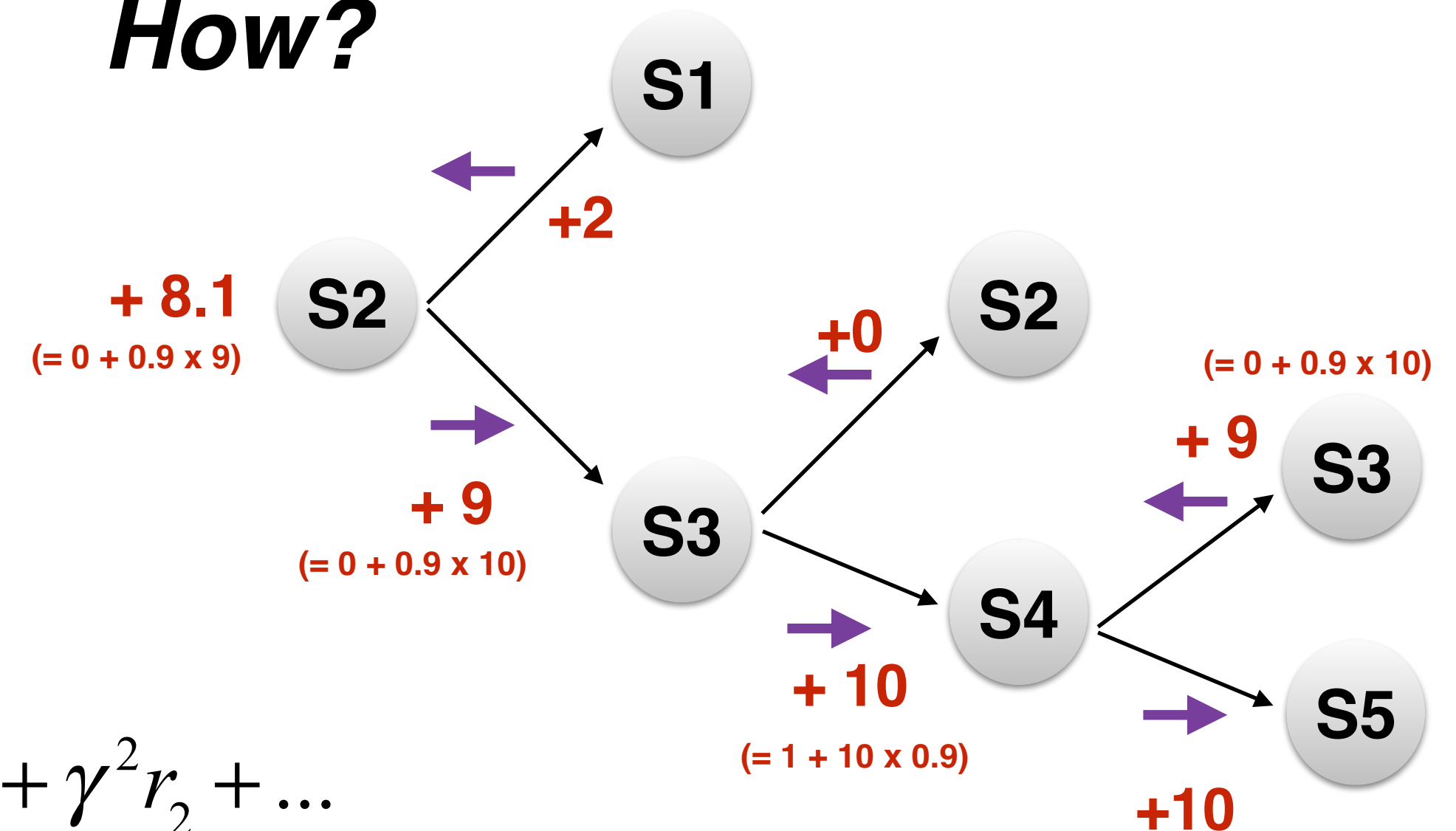
What do we want to learn?

how good is it to take action 1 in state 2

Q -table

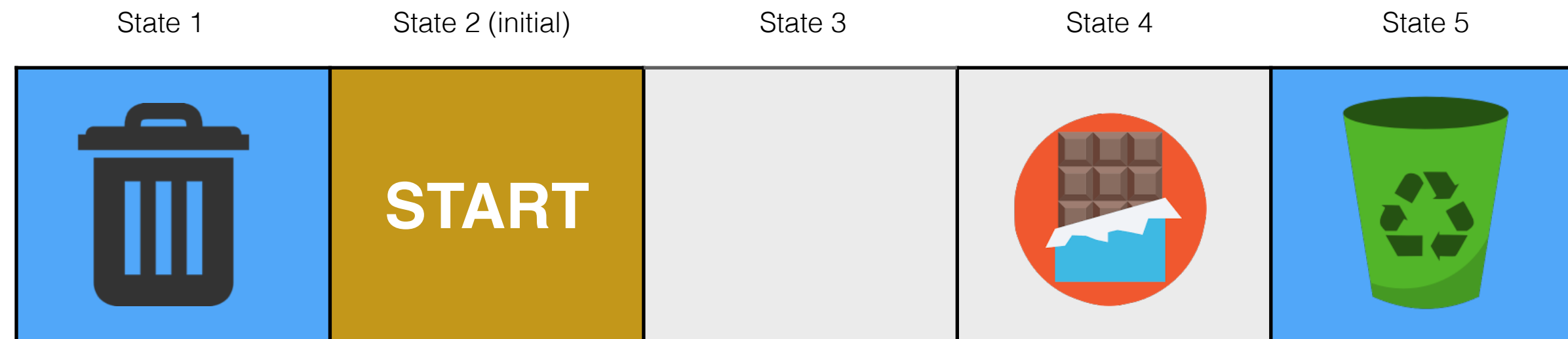
$Q = \begin{matrix} \begin{matrix} \xleftarrow{\text{\#actions}} \\ \begin{matrix} Q_{11} & Q_{12} \\ Q_{21} & Q_{22} \\ Q_{31} & Q_{32} \\ Q_{41} & Q_{42} \\ Q_{51} & Q_{52} \end{matrix} \\ \xrightarrow{\text{\#states}} \end{matrix} \end{matrix}$

How?

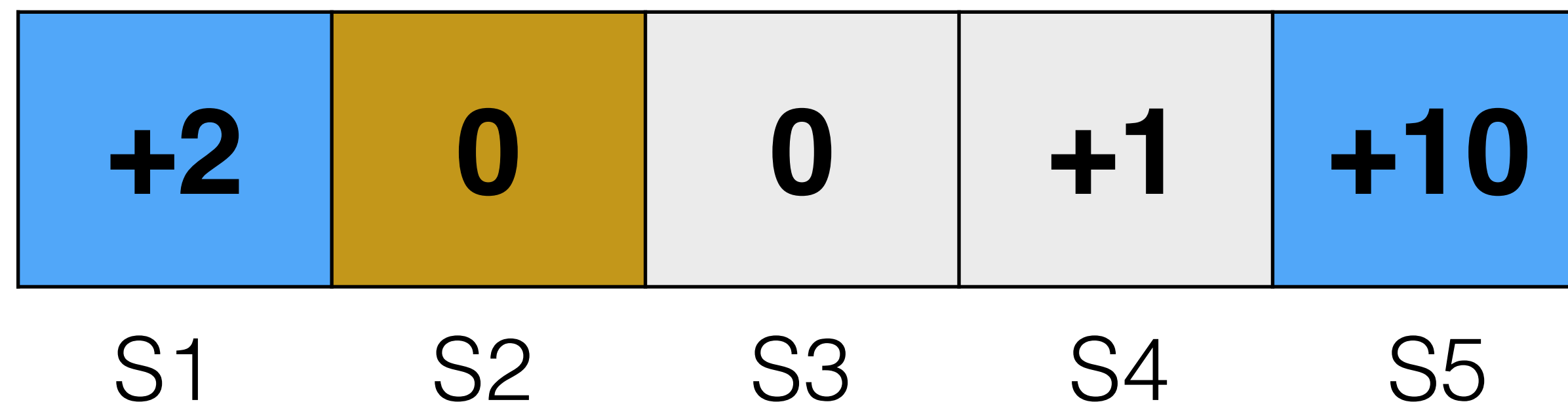


II. Recycling is good: an introduction to RL

Problem statement



Define reward “ r ” in every state



Assuming $\gamma = 0.9$

Discounted return $R = \sum_{t=0} \gamma^t r_t = r_0 + \gamma r_1 + \gamma^2 r_2 + \dots$

What do we want to learn?

how good is it to take action 1 in state 2

Q -table

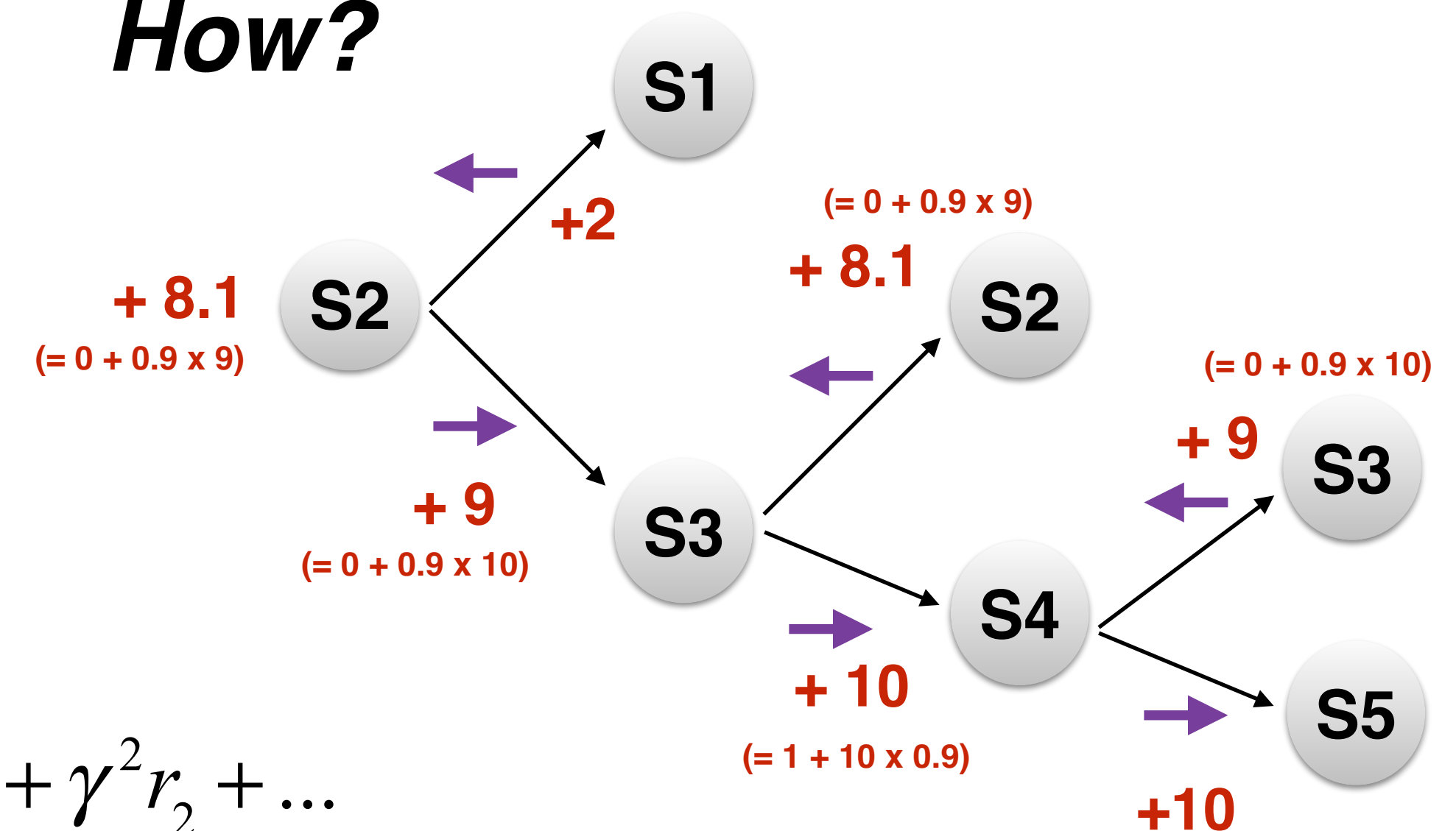
Diagram illustrating the Q -table structure:

#actions (horizontal axis)

#states (vertical axis)

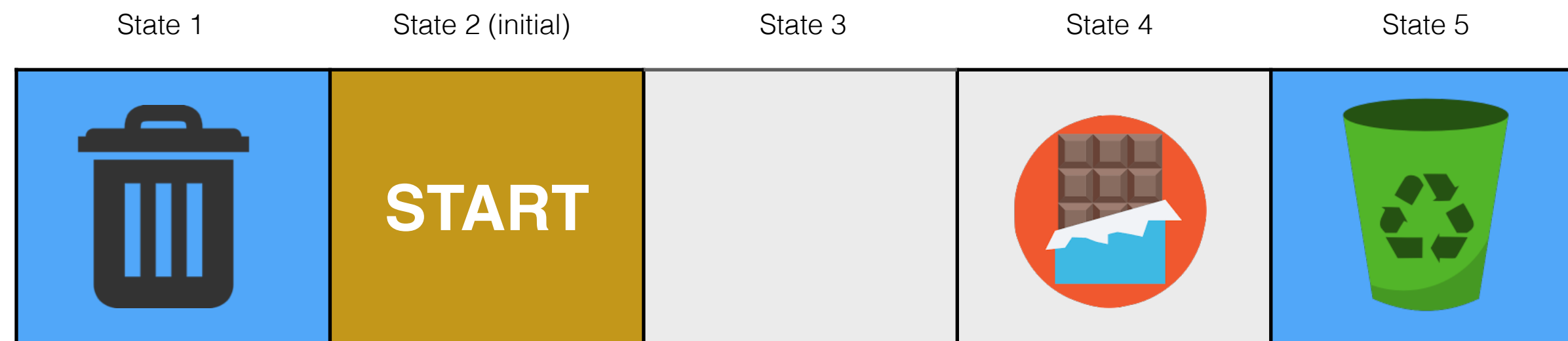
$$Q = \begin{pmatrix} Q_{11} & Q_{12} \\ Q_{21} & Q_{22} \\ Q_{31} & Q_{32} \\ Q_{41} & Q_{42} \\ Q_{51} & Q_{52} \end{pmatrix}$$

How?

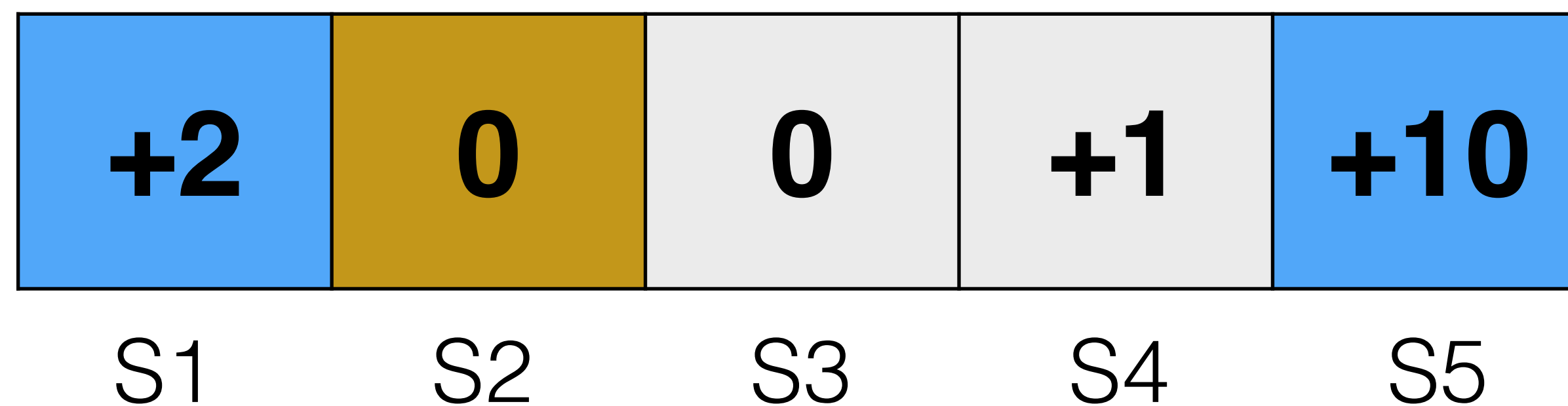


II. Recycling is good: an introduction to RL

Problem statement



Define reward “**r**” in every state



Assuming $\gamma = 0.9$

Discounted return $R = \sum_{t=0} \gamma^t r_t = r_0 + \gamma r_1 + \gamma^2 r_2 + \dots$

What do we want to learn?

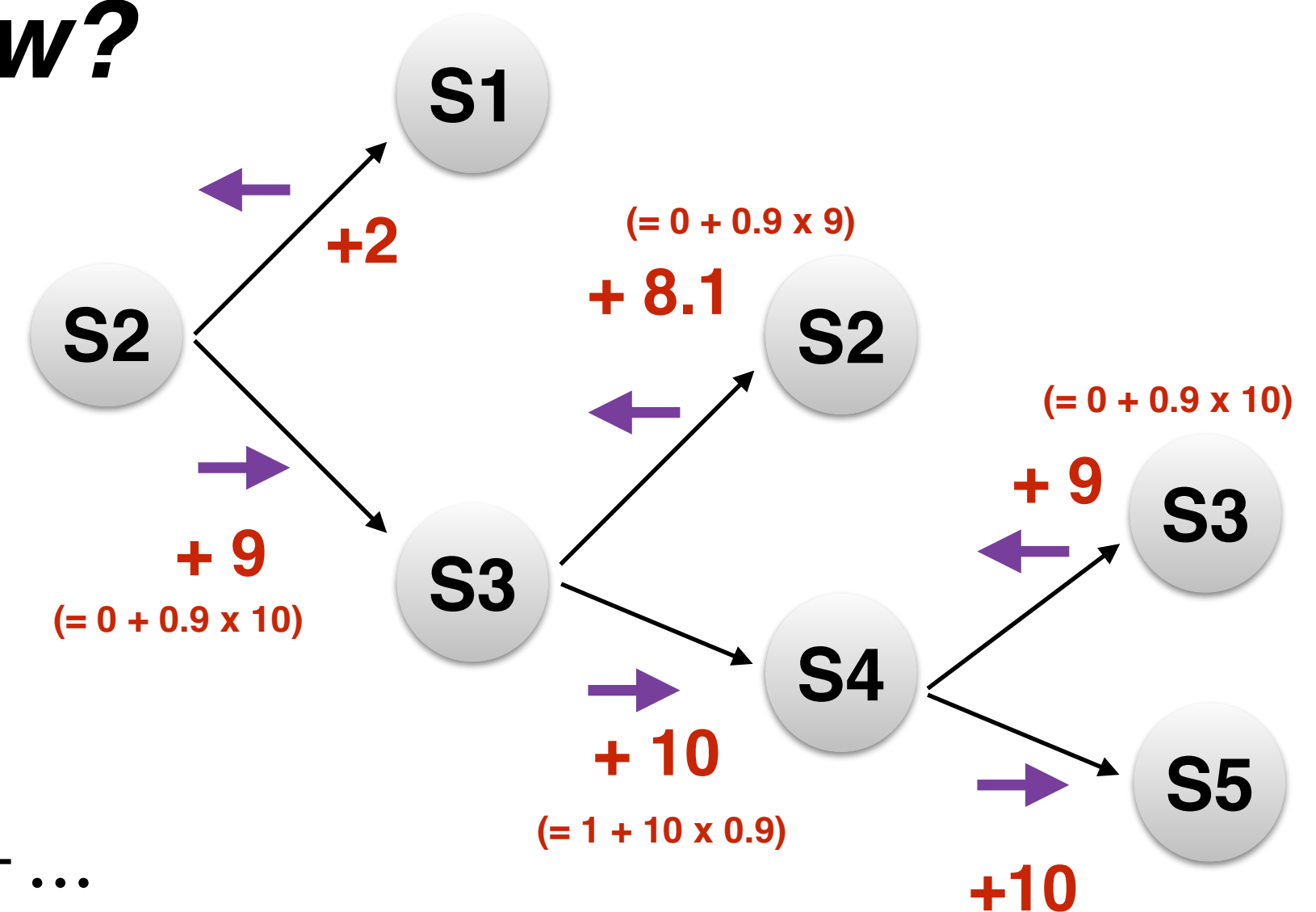
how good is it to take
action 1 in state 2

Q-table

$Q =$

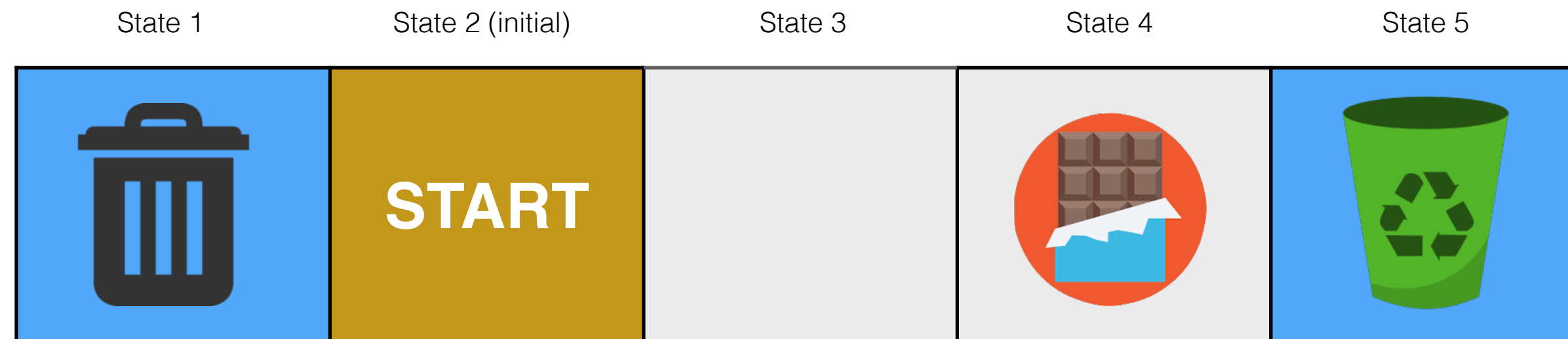
#actions		#states
0	0	
2	9	
8.1	10	
9	10	
0	0	

How?

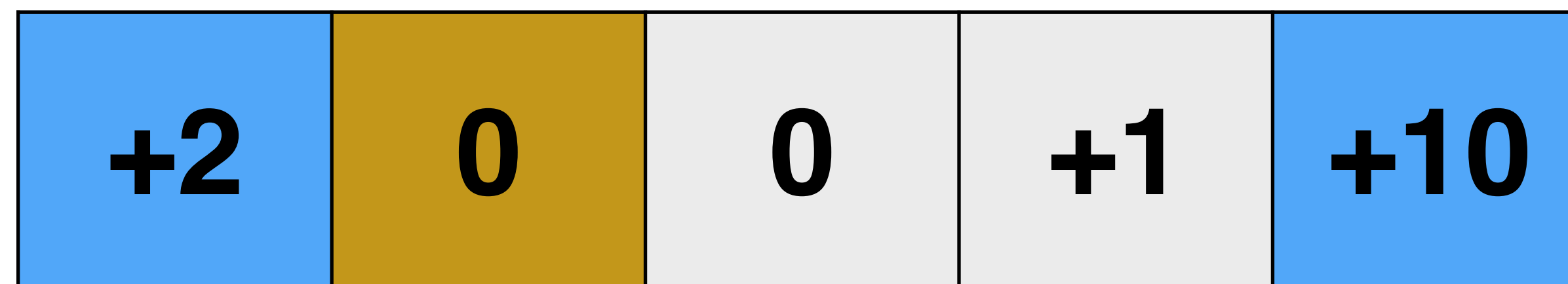


II. Recycling is good: an introduction to RL

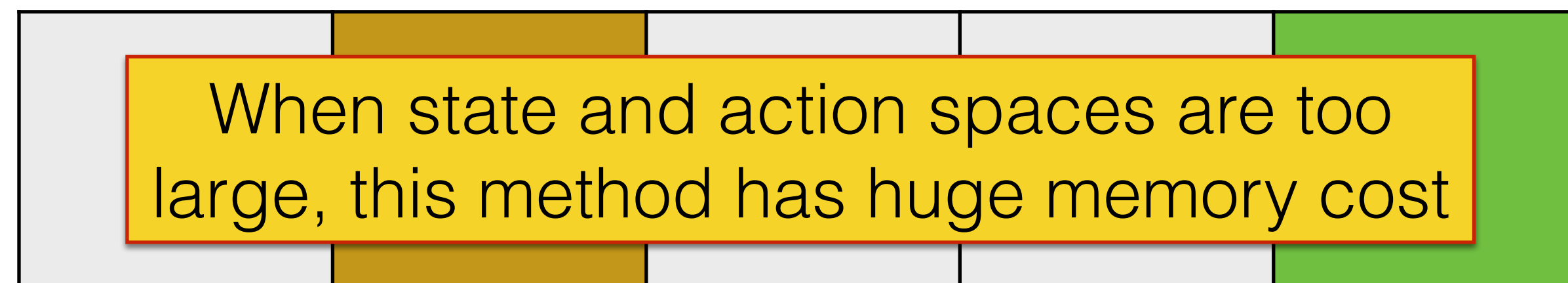
Problem statement



Define reward “ r ” in every state



Best strategy to follow if $\gamma = 0.9$



Function telling us our best strategy

What do we want to learn?

how good is it to take
action 1 in state 2

Q-table

$Q =$

#actions	
0	0
2	9
8.1	10
9	10
0	0

#states

*Bellman equation
(optimality equation)*

$$Q^*(s, a) = r + \gamma \max_{a'} (Q^*(s', a'))$$

Policy $\pi(s) = \arg \max_a (Q^*(s, a))$

Kian Katanforoosh

What we've learned so far:

- *Vocabulary: environment, agent, state, action, reward, total return, discount factor.*
- *Q-table: matrix of entries representing “how good is it to take action a in state s ”*
- *Policy: function telling us what's the best strategy to adopt*
- *Bellman equation satisfied by the optimal Q-table*

Today's outline

I. Motivation

II. Recycling is good: an introduction to RL

III. Deep Q-Learning

IV. Application of Deep Q-Learning: Breakout (Atari)

V. Tips to train Deep Q-Network

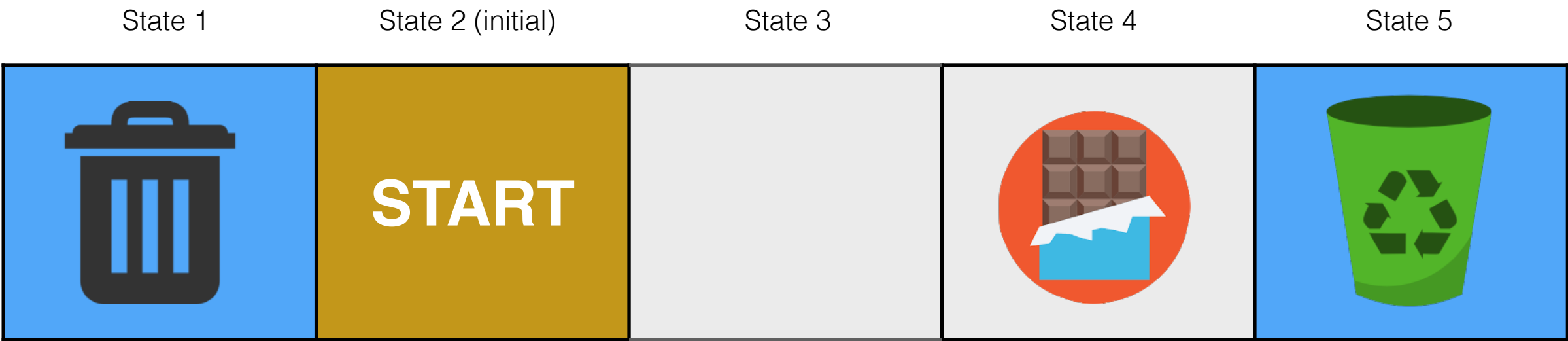
VI. Advanced topics

III. Deep Q-Learning

Main idea: find a Q-function to replace the Q-table

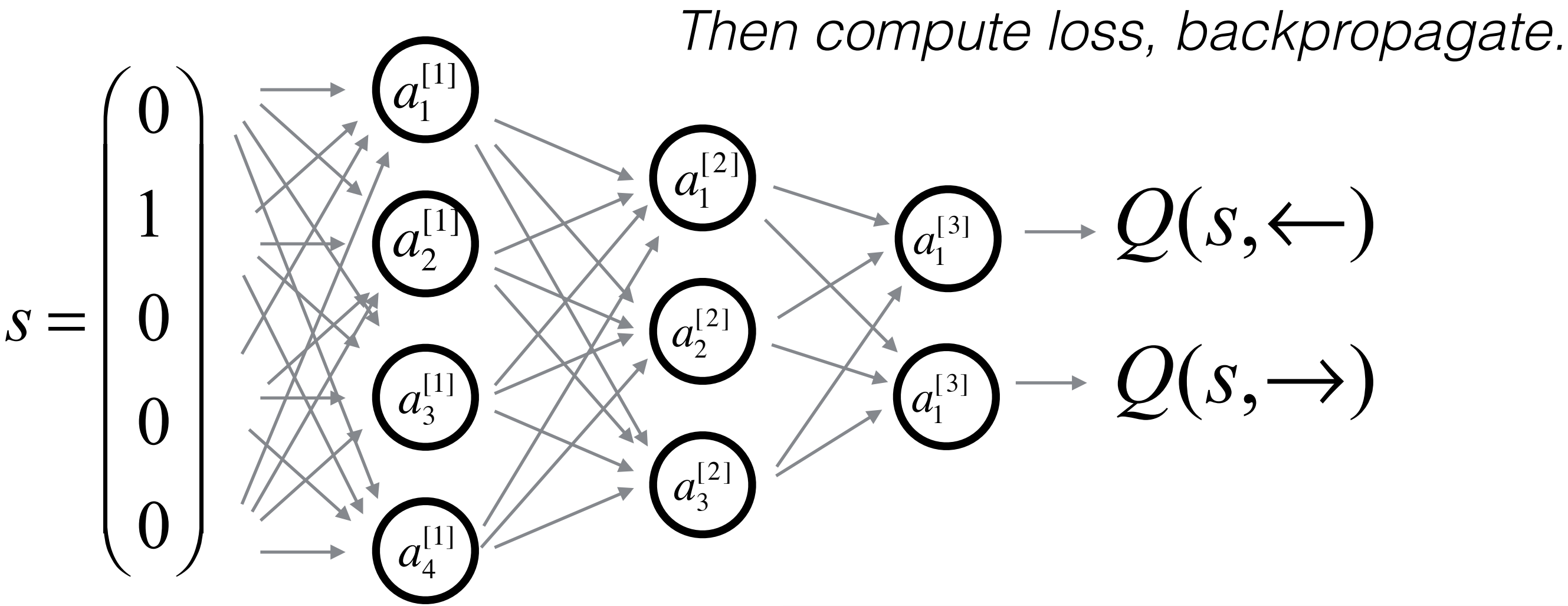
Problem statement

Neural Network



Q-table

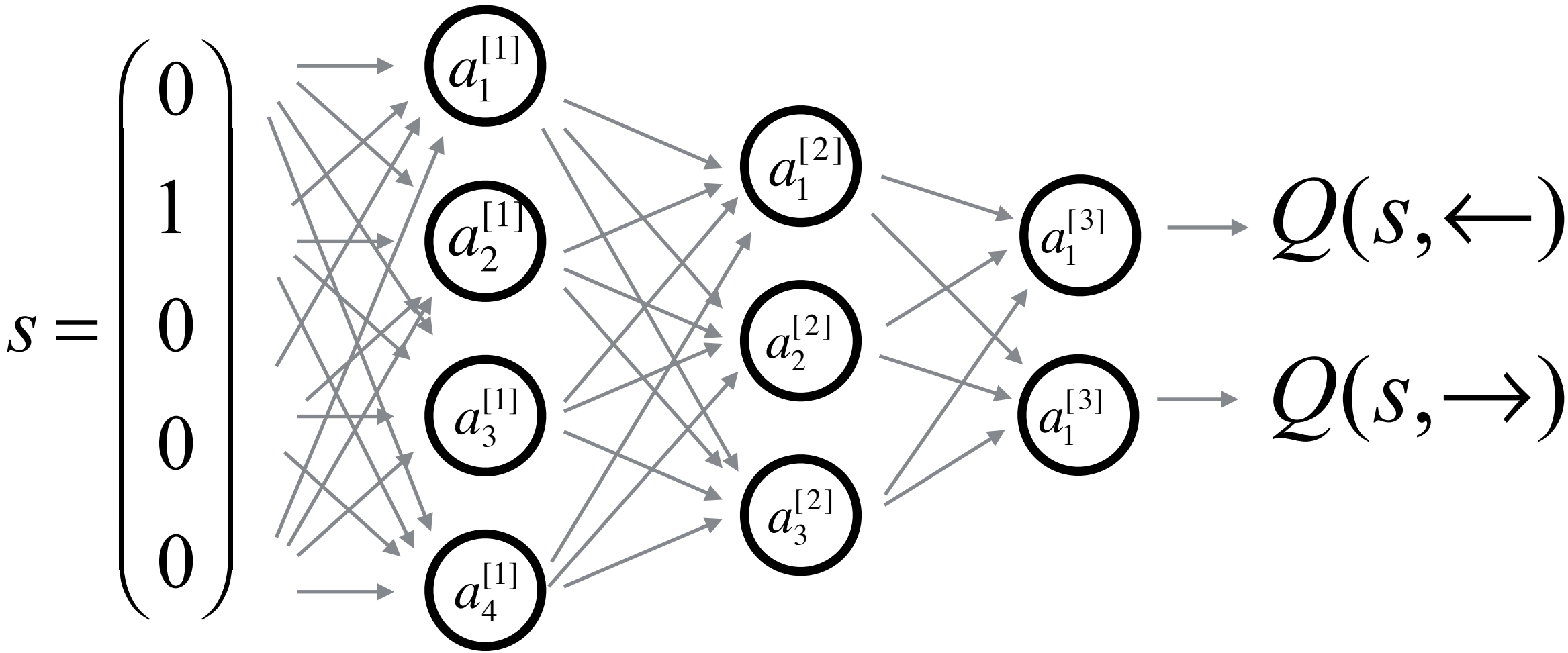
	#actions	
$Q =$	0	0
	2	9
	8.1	10
	9	10
	0	0
	#states	



How to compute the loss?

III. Deep Q-Learning

$$Q^*(s,a) = r + \gamma \max_{a'} (Q^*(s',a'))$$



Loss function

$$L = (y - Q(s, \leftarrow))^2$$

Target value

Case: $Q(s, \leftarrow) > Q(s, \rightarrow)$

$$y = r_{\leftarrow} + \gamma \max_{a'} (Q(s_{\leftarrow}^{next}, a'))$$

Hold fixed for backprop

Immediate reward for taking action $\leftarrow$ in state s

Discounted maximum future reward when you are in state $s_{\leftarrow}^{next}$

Case: $Q(s, \leftarrow) < Q(s, \rightarrow)$

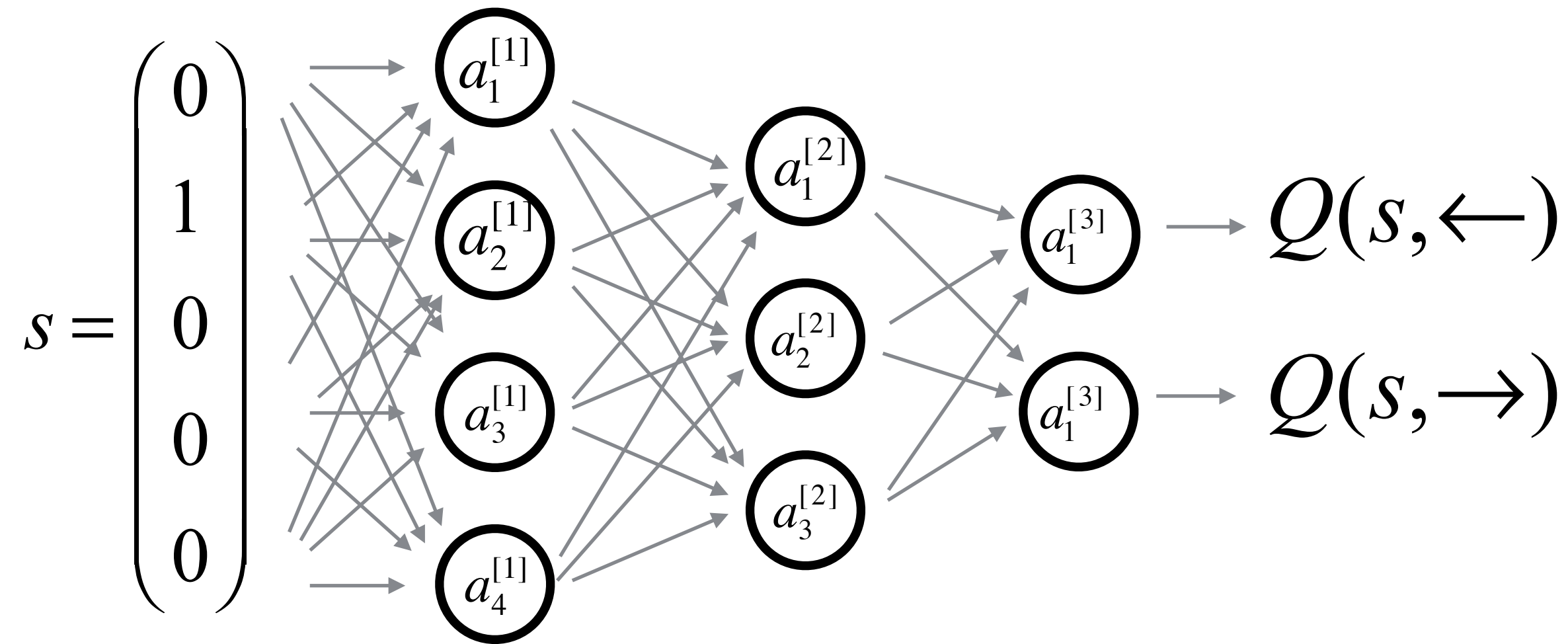
$$y = r_{\rightarrow} + \gamma \max_{a'} (Q(s_{\rightarrow}^{next}, a'))$$

Hold fixed for backprop

Immediate Reward for taking action $\rightarrow$ in state s

Discounted maximum future reward when you are in state $s_{\rightarrow}^{next}$

III. Deep Q-Learning



Loss function (regression)

$$L = (y - Q(s, \rightarrow))^2$$

Target value

Case: $Q(s, \leftarrow) > Q(s, \rightarrow)$

$$y = r_{\leftarrow} + \gamma \max_{a'} (Q(s_{\leftarrow}^{next}, a'))$$

Case: $Q(s, \leftarrow) < Q(s, \rightarrow)$

$$y = r_{\rightarrow} + \gamma \max_{a'} (Q(s_{\rightarrow}^{next}, a'))$$

Backpropagation

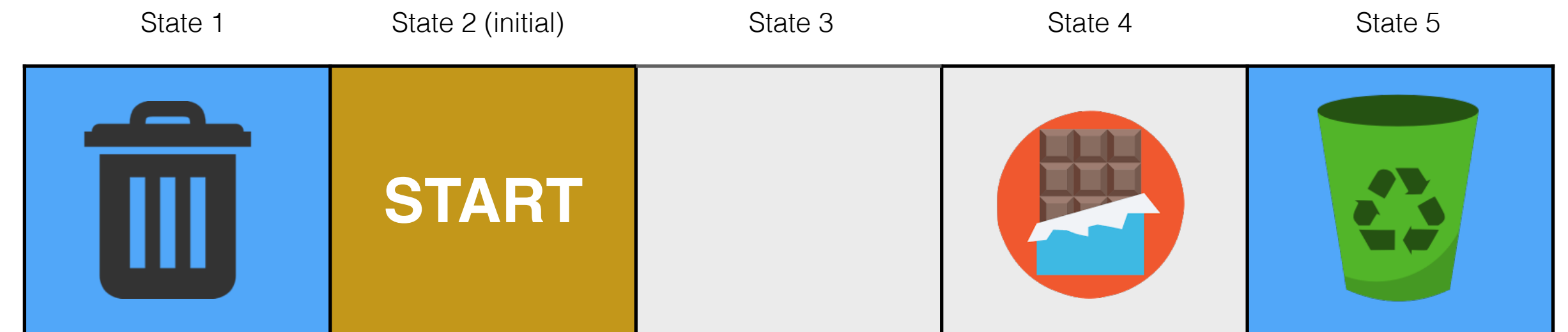
Compute $\frac{\partial L}{\partial W}$ and update W using stochastic gradient descent

Recap'

$$y = r_{\leftarrow} + \gamma \max_{a'} (Q(s_{\leftarrow}^{next}, a'))$$

DQN Implementation:

- Initialize your Q-network parameters
- Loop over episodes:
 - Start from initial state s
 - Loop over time-steps:
 - Forward propagate s in the Q-network
 - Execute action a (that has the maximum $Q(s,a)$ output of Q-network)
 - Observe reward r and next state s'
 - Compute targets y by forward propagating state s' in the Q-network, then compute loss.
 - Update parameters with gradient descent



Today's outline

I. Motivation

II. Recycling is good: an introduction to RL

III. Deep Q-Networks

IV. Application of Deep Q-Network: Breakout (Atari)

V. Tips to train Deep Q-Network

VI. Advanced topics

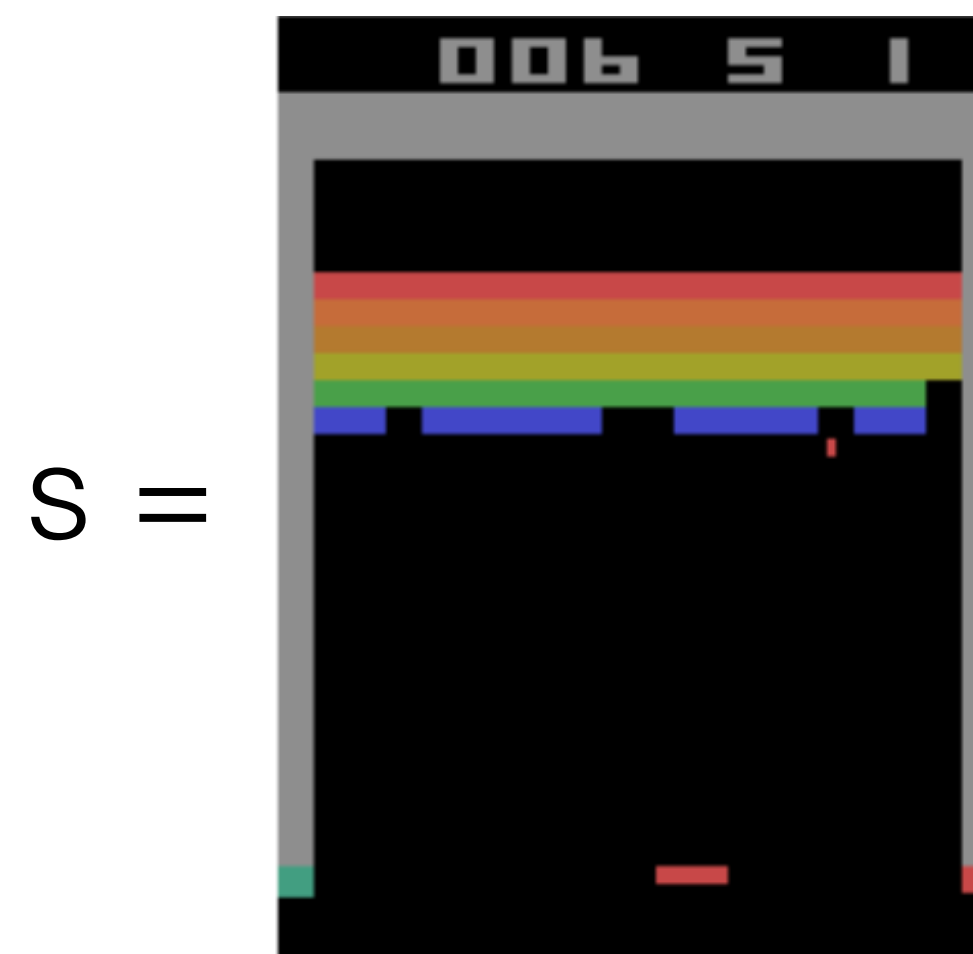
IV. Deep Q-Learning application: Breakout (Atari)

Goal: play breakout, i.e. destroy all the bricks.

Demo



input of Q-network



Output of Q-network

Q-values

$$\begin{pmatrix} Q(s, \leftarrow) \\ Q(s, \rightarrow) \\ Q(s, -) \end{pmatrix}$$

Would that work?

[Video credits to Two minute papers: Google DeepMind's Deep Q-learning playing Atari Breakout <https://www.youtube.com/watch?v=V1eYniJ0Rnk>]

[Mnih, Kavukcuoglu, Silver et al. (2015): Human Level Control through Deep Reinforcement Learning]

Kian Katanforoosh

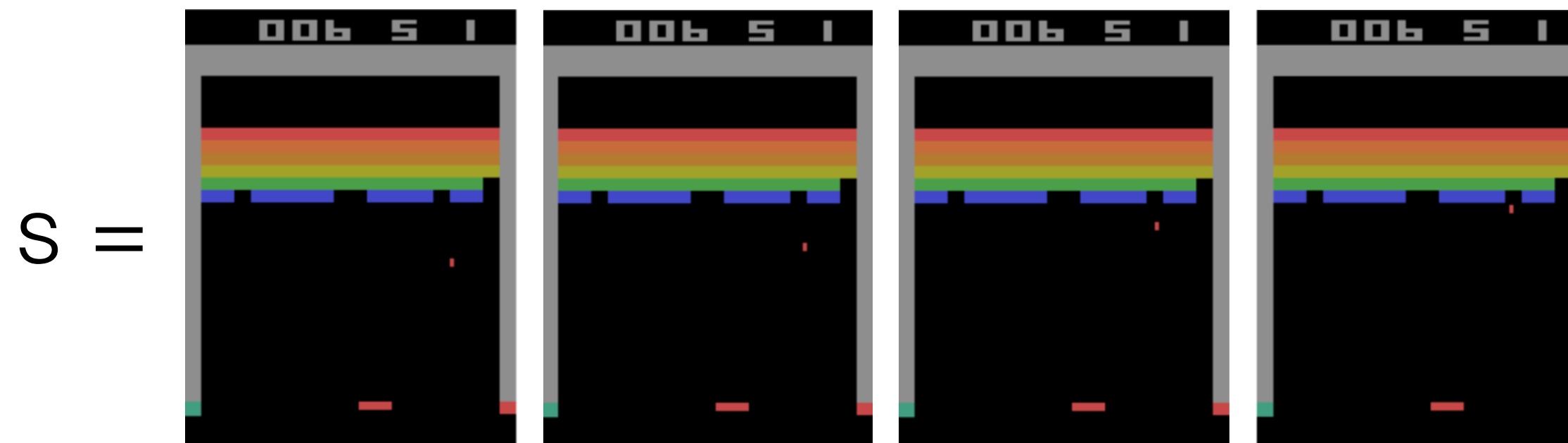
IV. Deep Q-Learning application: Breakout (Atari)

Goal: play breakout, i.e. destroy all the bricks.

Demo



input of Q-network

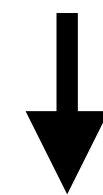


Output of Q-network

Q-values

$$\begin{pmatrix} Q(s, \leftarrow) \\ Q(s, \rightarrow) \\ Q(s, -) \end{pmatrix}$$

Preprocessing



$\phi(s)$

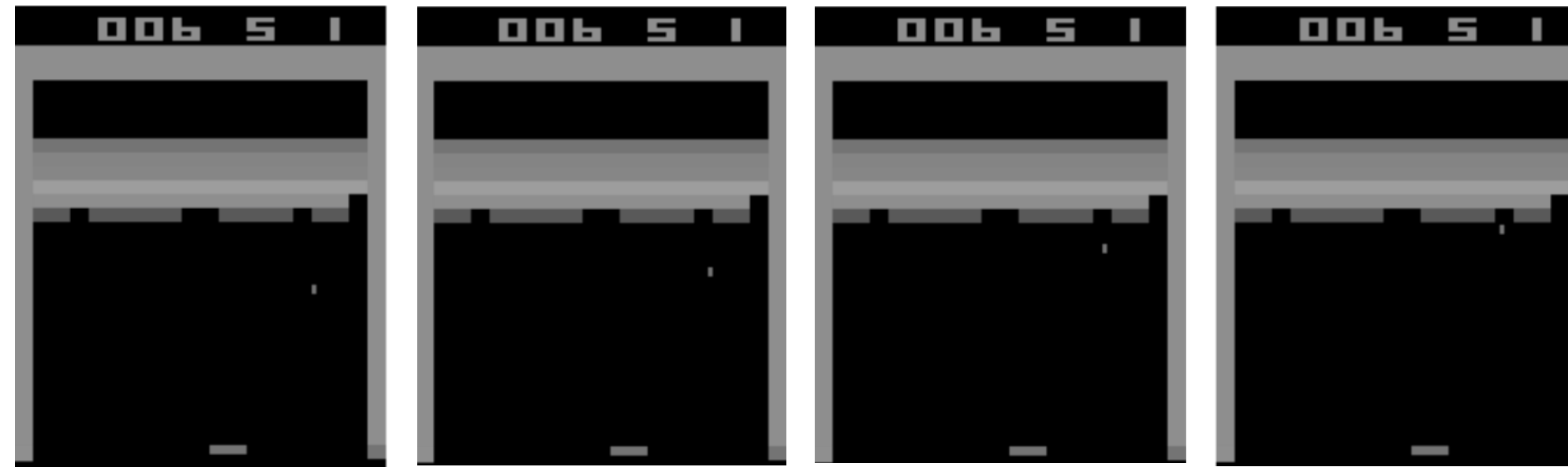
- Convert to grayscale
- Reduce dimensions (h,w)
- History (4 frames)

What is done in preprocessing?

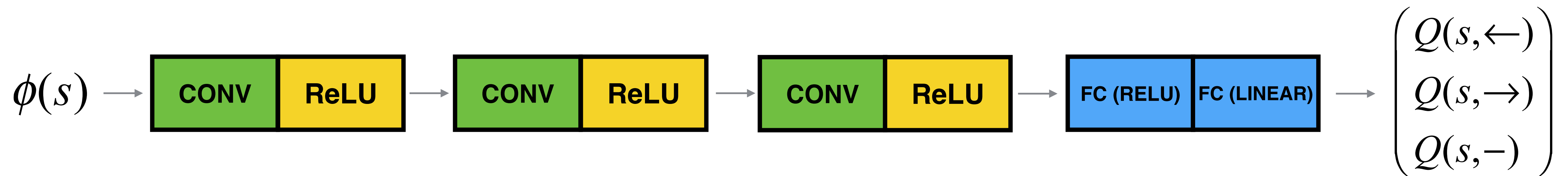
IV. Deep Q-Learning application: Breakout (Atari)

input of Q-network

$\phi(s) =$



Deep Q-network architecture?



Recap' (+ preprocessing + terminal state)

DQN Implementation:

- Initialize your Q-network parameters
- Loop over episodes:
 - Start from initial state ~~s~~ $\phi(s)$
 - Loop over time-steps: $\phi(s)$
 - Forward propagate ~~s~~ in the Q-network $\phi(s)$
 - Execute action a (that has the maximum $Q(\phi(s), a)$ output of Q-network)
 - Observe reward r and next state s'
 - **Use s' to create $\phi(s')$**
 - Compute targets y by forward propagating state ~~s'~~ $\phi(s')$ in the Q-network, then compute loss.
 - Update parameters with gradient descent

Some training challenges:

- Keep track of terminal step
- Experience replay
- Epsilon greedy action choice (Exploration / Exploitation tradeoff)

Recap' (+ preprocessing + terminal state)

DQN Implementation:

- Initialize your Q-network parameters
- Loop over episodes:
 - Start from initial state ~~s~~ $\phi(s)$
 - Loop over time-steps: $\phi(s)$
 - Forward propagate ~~s~~ in the Q-network $\phi(s)$
 - Execute action a (that has the maximum $Q(\phi(s), a)$ output of Q-network)
 - Observe reward r and next state s'
 - **Use s' to create $\phi(s')$**
 - Compute targets y by forward propagating state ~~s'~~ $\phi(s')$ in the Q-network, then compute loss.
 - Update parameters with gradient descent

Some training challenges:

- Keep track of terminal step
- Experience replay
- Epsilon greedy action choice (Exploration / Exploitation tradeoff)

Recap' (+ preprocessing + terminal state)

DQN Implementation:

- Initialize your Q-network parameters
- Loop over episodes:
 - Start from initial state ~~s~~ $\phi(s)$
 - **Create a boolean to detect terminal states: $terminal = False$**
 - Loop over time-steps:
 - Forward propagate ~~s~~ in the Q-network $\phi(s)$
 - Execute action a (that has the maximum $Q(\text{ ~~$s$~~ , a)}$ output of Q-network)
 - Observe reward r and next state s'
 - Use s' to create $\phi(s')$
 - **Check if s' is a terminal state.** Compute targets y by forward propagating state ~~s~~ in the Q-network, then compute loss.
 - Update parameters with gradient descent

Some training challenges:

- Keep track of terminal step
- Experience replay
- Epsilon greedy action choice (Exploration / Exploitation tradeoff)

$$\begin{cases} \text{if } terminal = False & : y = r + \gamma \max_{a'} (Q(s', a')) \\ \text{if } terminal = True & : y = r \quad (break) \end{cases}$$

$\phi(s')$

IV - DQN training challenges

Experience replay

1 experience (leads to one iteration of gradient descent)

Current method is to start from initial state s and follow:

E1 $\phi(s) \rightarrow a \rightarrow r \rightarrow \phi(s')$
E2 $\phi(s') \rightarrow a' \rightarrow r' \rightarrow \phi(s'')$
E3 $\phi(s'') \rightarrow a'' \rightarrow r'' \rightarrow \phi(s''')$
...

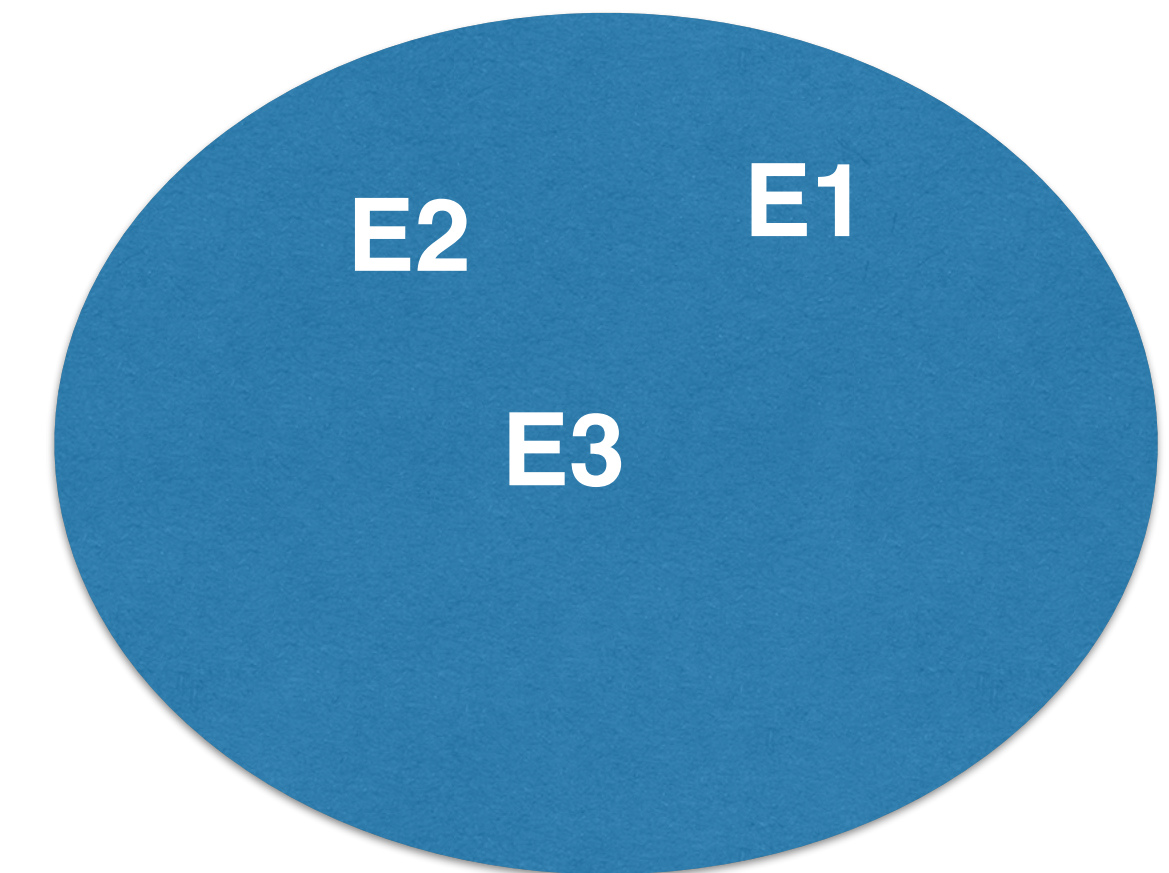
Experience Replay

E1

E2

E3

...



Replay memory (D)

Training: $E1 \rightarrow E2 \rightarrow E3$

Training: $E1 \rightarrow \text{sample}(E1, E2) \rightarrow \text{sample}(E1, E2, E3) \rightarrow \text{sample}(E1, E2, E3, E4) \rightarrow \dots$

Can be used with mini batch gradient descent

Advantages of experience replay?

Recap' (+ experience replay)

DQN Implementation:

- Initialize your Q-network parameters
- **Initialize replay memory D**
- Loop over episodes:
 - Start from initial state $\phi(s)$
 - Create a boolean to detect terminal states: $\text{terminal} = \text{False}$
 - Loop over time-steps:
 - Forward propagate $\phi(s)$ in the Q-network
 - Execute action a (that has the maximum $Q(\phi(s), a)$ output of Q-network)
 - Observe reward r and next state s'
 - Use s' to create $\phi(s')$
 - **Add experience $(\phi(s), a, r, \phi(s'))$ to replay memory (D)**
- **Sample random mini-batch of transitions from D**
 - Check if s' is a terminal state. Compute targets y by forward propagating state $\phi(s')$ in the Q-network, then compute loss.
 - Update parameters with gradient descent

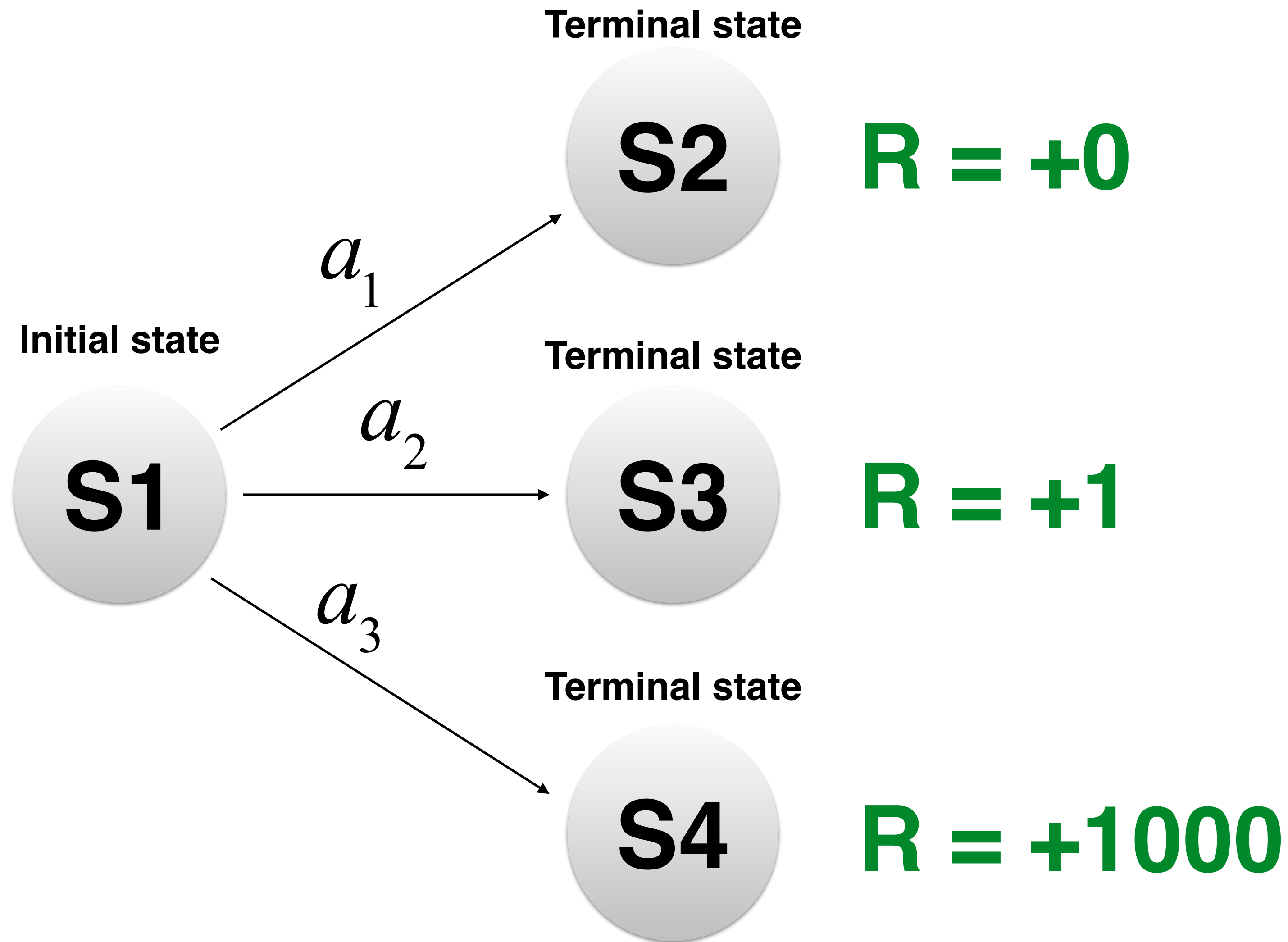
Some training challenges:

- Keep track of terminal step
- Experience replay
- Epsilon greedy action choice (Exploration / Exploitation tradeoff)

The transition resulting from this is added to D , and will not necessarily be used in this iteration's update!

Update using sampled transitions

Exploration vs. Exploitation



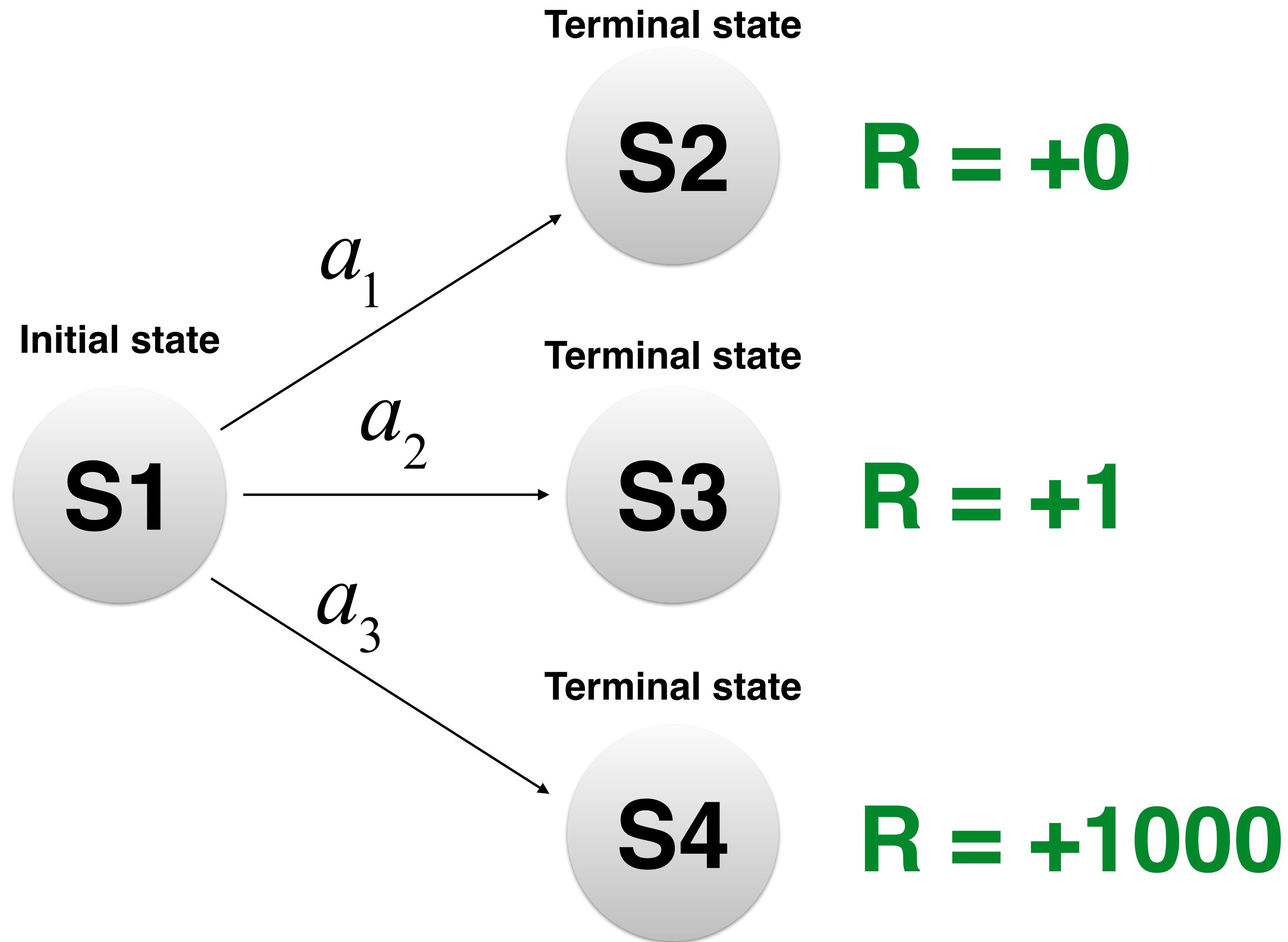
Just after initializing the Q-network, we get:

$$Q(S1, a_1) = 0.5$$

$$Q(S1, a_2) = 0.4$$

$$Q(S1, a_3) = 0.3$$

Exploration vs. Exploitation



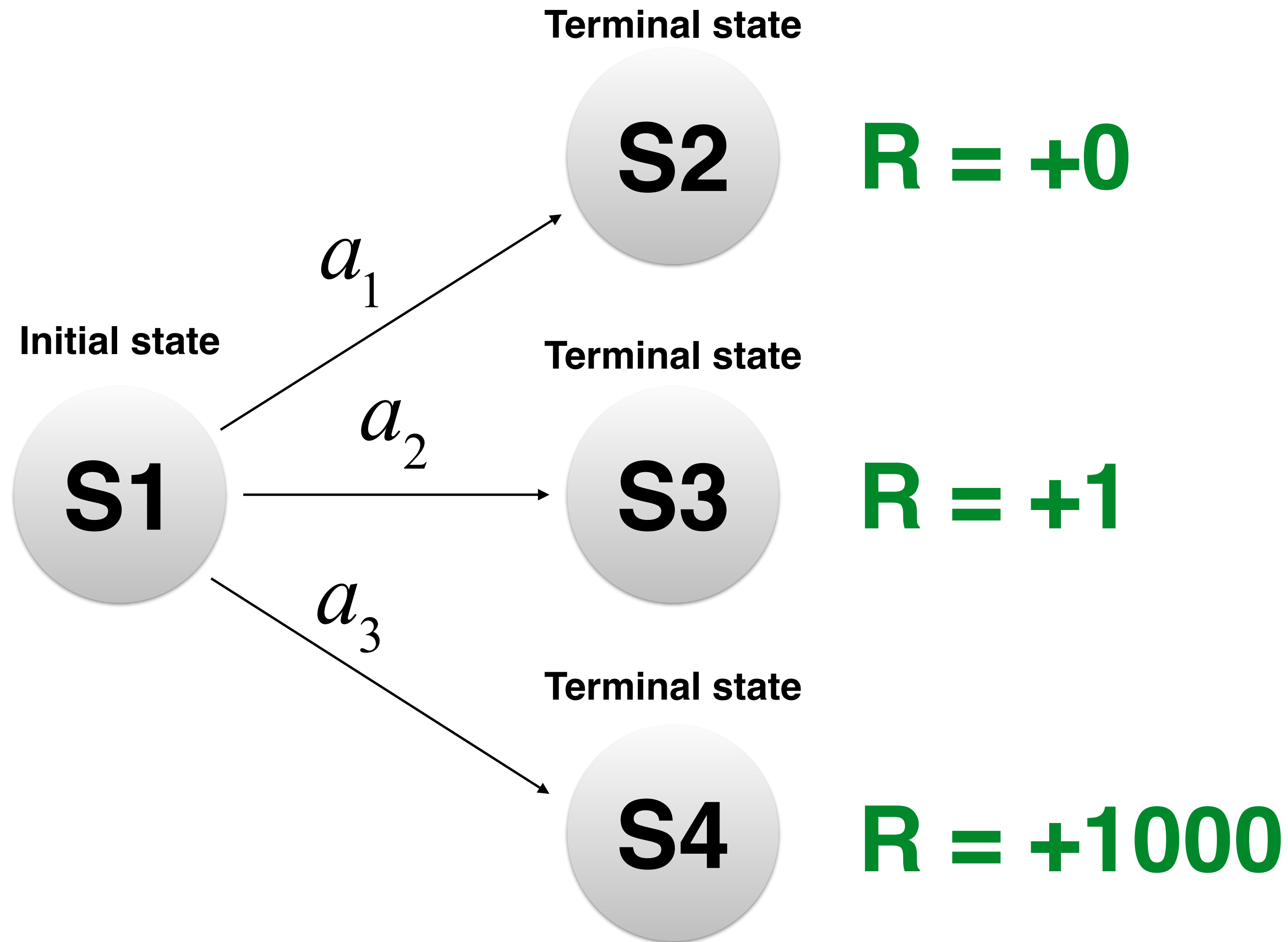
Just after initializing the Q-network, we get:

$$Q(S1, a_1) = \cancel{0.5} \quad 0$$

$$Q(S1, a_2) = 0.4$$

$$Q(S1, a_3) = 0.3$$

Exploration vs. Exploitation



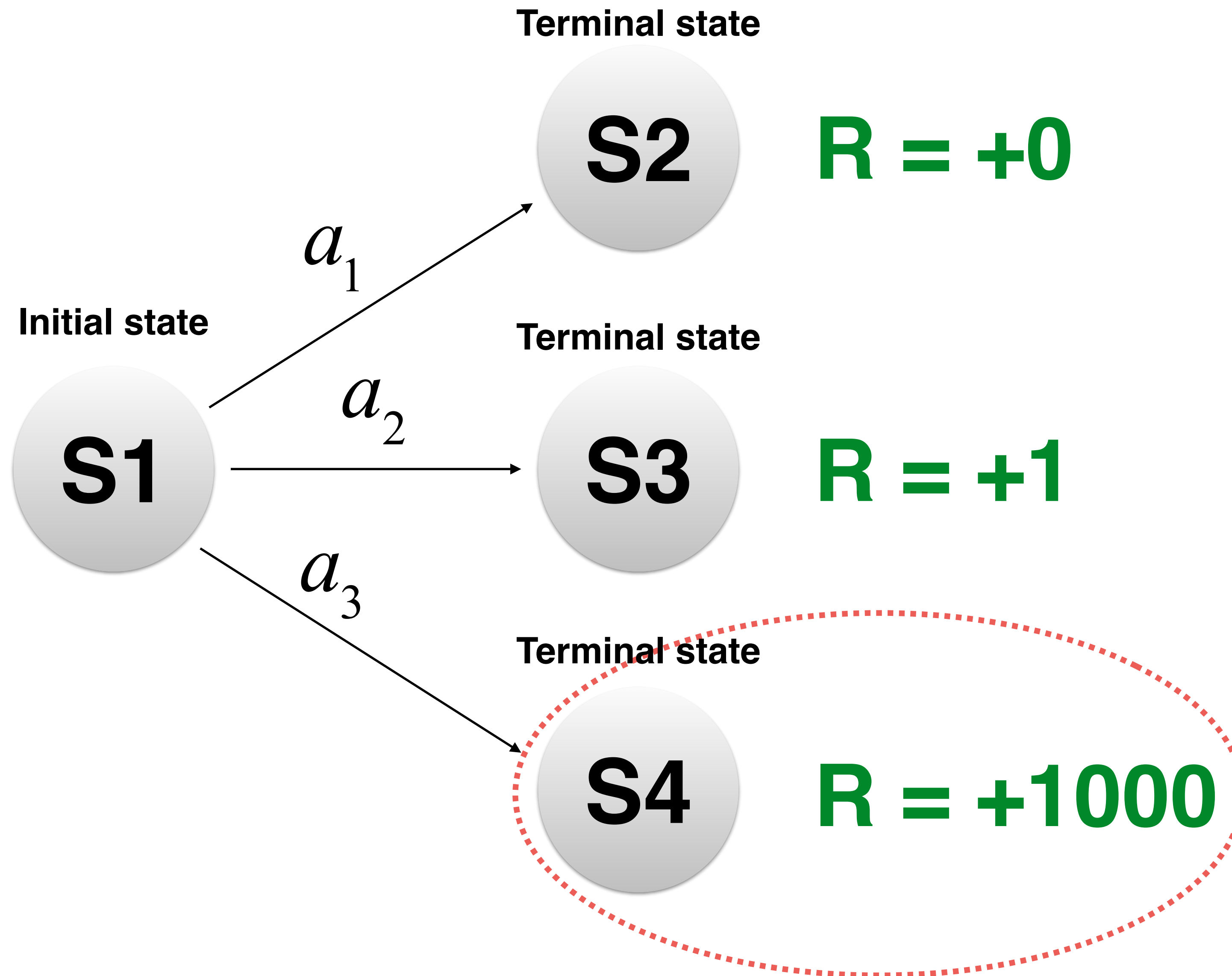
Just after initializing the Q-network, we get:

$$Q(S1, a_1) = \cancel{0.5} \quad 0$$

$$Q(S1, a_2) = \cancel{0.4} \quad 1$$

$$Q(S1, a_3) = 0.3$$

Exploration vs. Exploitation



Just after initializing the Q-network, we get:

$$Q(S1, a_1) = \cancel{0.5} \quad 0$$

$$Q(S1, a_2) = \cancel{0.4} \quad 1$$

$$Q(S1, a_3) = 0.3$$

Will never be visited, because
 $Q(S1, a_3) < Q(S1, a_2)$

Recap' (+ epsilon greedy action)

DQN Implementation:

- Initialize your Q-network parameters
- Initialize replay memory D
- Loop over episodes:
 - Start from initial state $\phi(s)$
 - Create a boolean to detect terminal states: $terminal = False$
 - Loop over time-steps:
 - **With probability epsilon, take random action a .**
 - **Otherwise:**
 - Forward propagate $\phi(s)$ in the Q-network
 - Execute action a (that has the maximum $Q(\phi(s), a)$ output of Q-network).
 - Observe reward r and next state s'
 - Use s' to create $\phi(s')$
 - Add experience $(\phi(s), a, r, \phi(s'))$ to replay memory (D)
 - Sample random mini-batch of transitions from D
 - Check if s' is a terminal state. Compute targets y by forward propagating state $\phi(s')$ in the Q-network, then compute loss.
 - Update parameters with gradient descent

Overall recap'

DQN Implementation:

- Initialize your Q-network parameters
- **Initialize replay memory D**
- Loop over episodes:
 - Start from initial state $\phi(s)$
 - **Create a boolean to detect terminal states: $terminal = False$**
 - Loop over time-steps:
 - **With probability ϵ , take random action a .**
 - **Otherwise:**
 - Forward propagate $\phi(s)$ in the Q-network
 - Execute action a (that has the maximum $Q(\phi(s), a)$ output of Q-network).
 - Observe rewards r and next state s'
 - **Use s' to create $\phi(s')$**
 - **Add experience $(\phi(s), a, r, \phi(s'))$ to replay memory (D)**
 - **Sample random mini-batch of transitions from D**
 - **Check if s' is a terminal state.** Compute targets y by forward propagating state $\phi(s')$ in the Q-network, then compute loss.
 - Update parameters with gradient descent

- **Preprocessing**
- **Detect terminal state**
- **Experience replay**
- **Epsilon greedy action**

Results



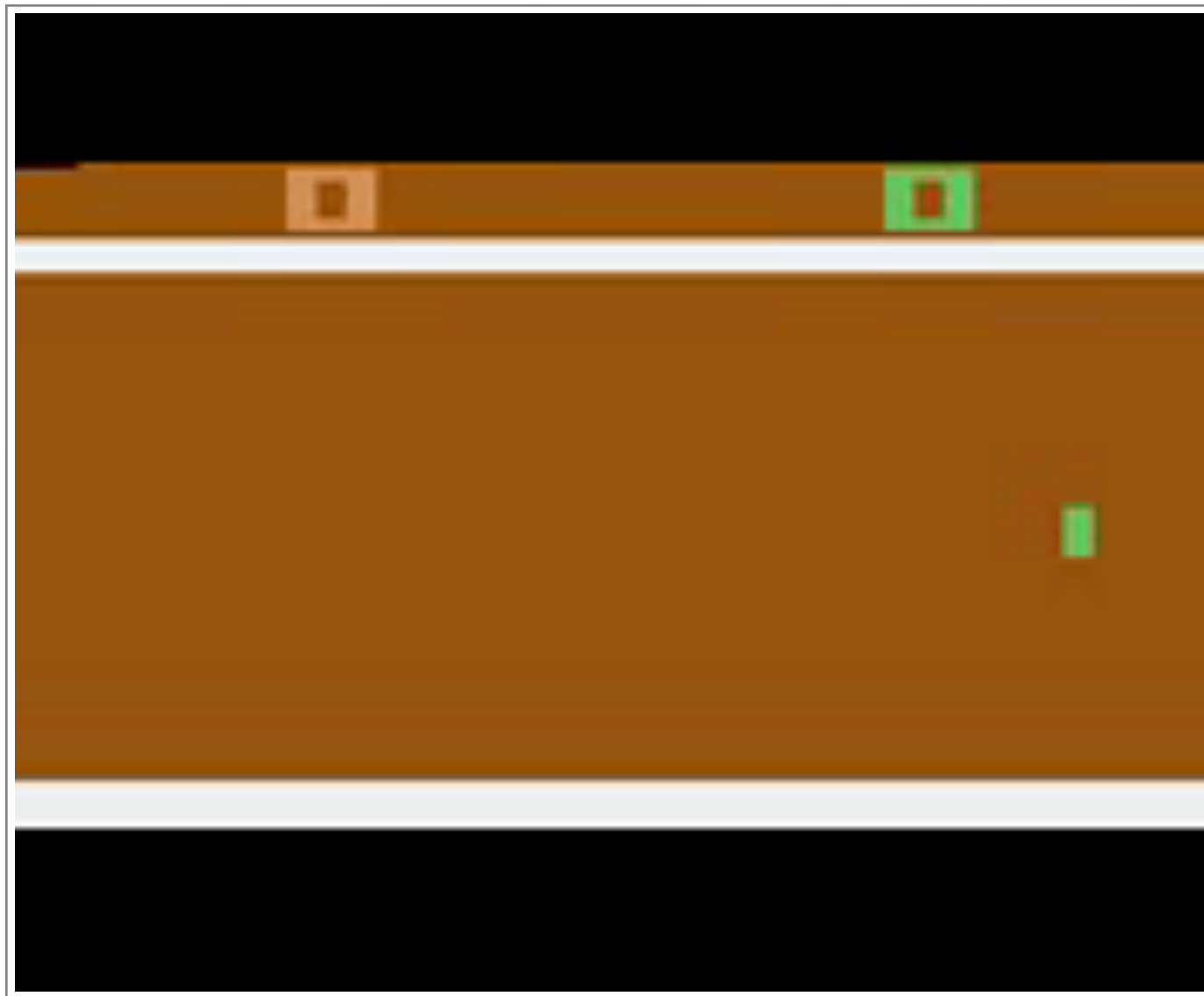
[Mnih, Kavukcuoglu, Silver et al. (2015): Human Level Control through Deep Reinforcement Learning]

[Credits: DeepMind, DQN Breakout - <https://www.youtube.com/watch?v=TmPfTpjtdgg>]

Kian Katanforoosh

Other Atari games

Pong



[Chia-Hsuan Lee, Atari Seaquest Double DQN Agent - <https://www.youtube.com/watch?v=NirMkC5uvWU>]

SeaQuest



[moooopan, Deep Q-Network Plays Atari 2600 Pong - https://www.youtube.com/watch?v=p88R2_3yWPA]

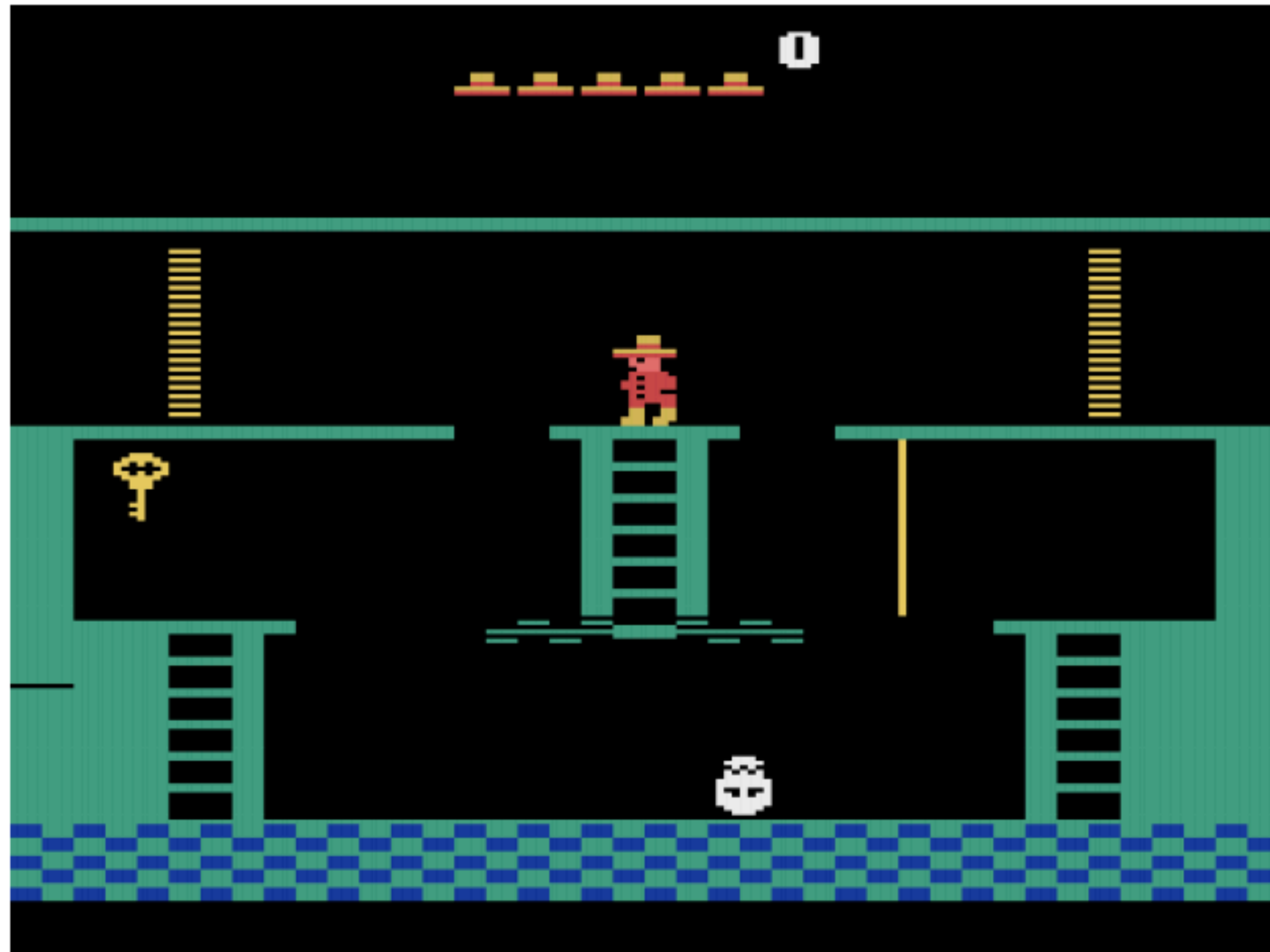
Space Invaders



[DeepMind: DQN SPACE INVADERS - <https://www.youtube.com/watch?v=W2CAghUiofY&t=2s>]

Difference between with and without human knowledge

Imitation learning is about learning by observing expert behavior.



[Source: Bellemare et al. (2016): Unifying Count-Based Exploration and Intrinsic Motivation]

RLHF: Humans rate or rank the outputs generated by the model, guiding the agent toward more preferred behaviors.



[Ouyang, Wu, Jiang, Almeida, Wainwright, Mishkin, et al. (2022): Training language models to follow instructions with human feedback]

[Ho et al. (2016): Generative Adversarial Imitation Learning]

Kian Katanforoosh

Today's outline

I. Motivation

II. Recycling is good: an introduction to RL

III. Deep Q-Networks

IV. Application of Deep Q-Network: Breakout (Atari)

V. Tips to train Deep Q-Network

VI. Advanced topics

VI - Advanced topics

	DQN	PPO
Algorithm Type	Value-based (learns Q-values)	<u>Policy-based (learns a policy directly)</u>
Suitable Action Space	Discrete actions only	Works well with both discrete and <u>continuous actions</u>
Core Objective	Maximize Q-values for optimal action selection	Maximize <u>expected advantage</u> while limiting abrupt policy updates
Update Mechanism	Uses Q-learning with Bellman equation to update Q-values	Uses policy gradient with <u>a clipped objective</u> to stabilize learning
Exploration Strategy	Epsilon-greedy (random actions with probability ϵ)	Probabilistic exploration inherent in policy gradients
Training Stability	Can be unstable; needs techniques like experience replay	Generally more stable; clipping and advantage function provide stability
Action Selection	Chooses action with the highest Q-value in a given state	<u>Samples actions based on probabilities output by the policy</u>
Value Function	Q-function, $Q(s, a)$, for each state-action pair	<u>Value function, $V(s)$, with advantage function $A(s, a)$</u>
Loss Function	Mean squared error between predicted and target Q-values (Bellman Equation)	Clipped surrogate objective function to limit large updates
Implementation Complexity	Relatively simpler but limited to discrete actions	More complex but flexible for various action spaces
Example Use Case	Game-playing with a small action space, like Atari games	Robotics control, continuous environments, and complex tasks

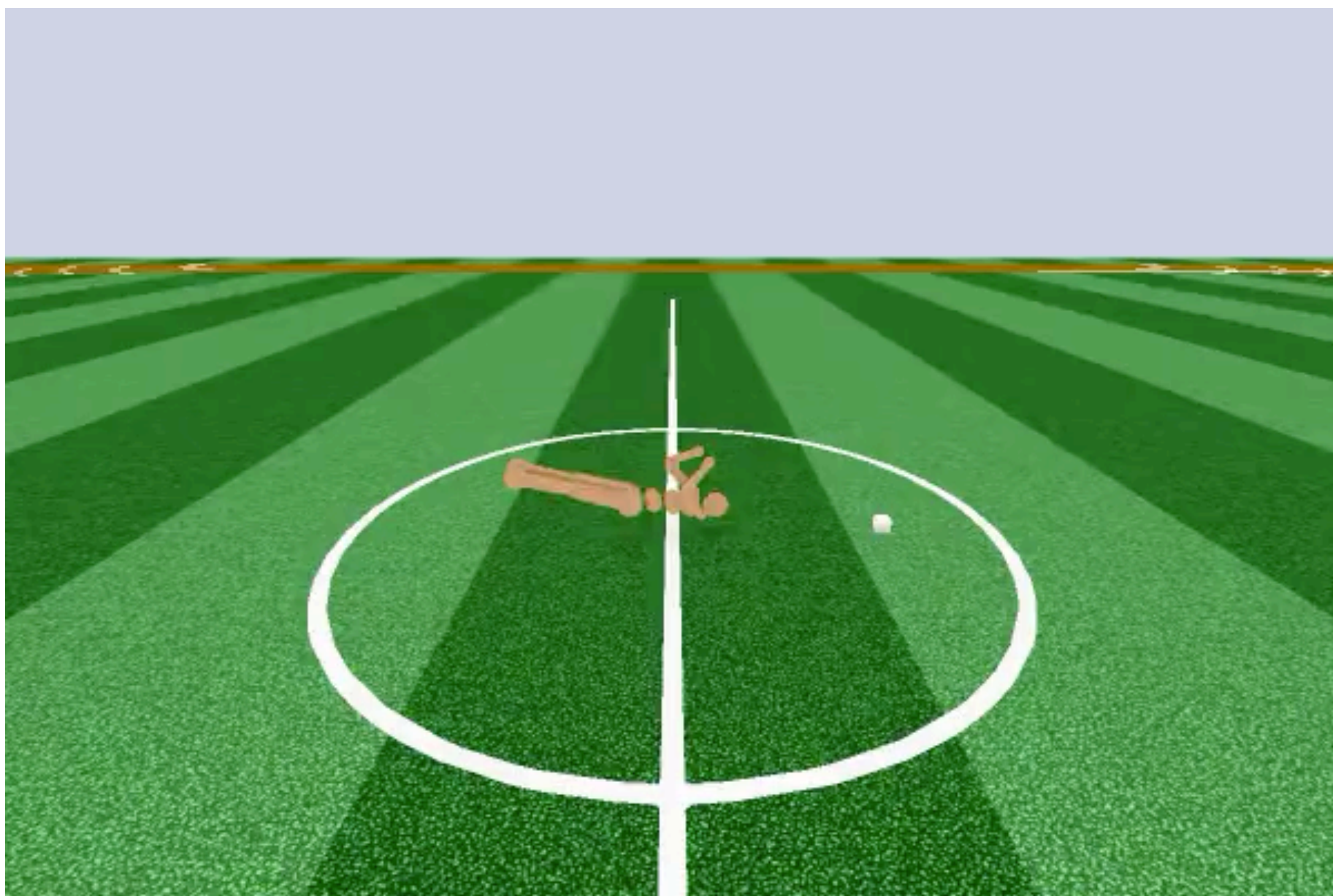
[Schulman et al. (2017): Trust Region Policy Optimization]

[Schulman et al. (2017): Proximal Policy Optimization]

VI - Advanced topics

Policy Gradient Methods

PPO



[[Open AI Blog](#)]

TRPO

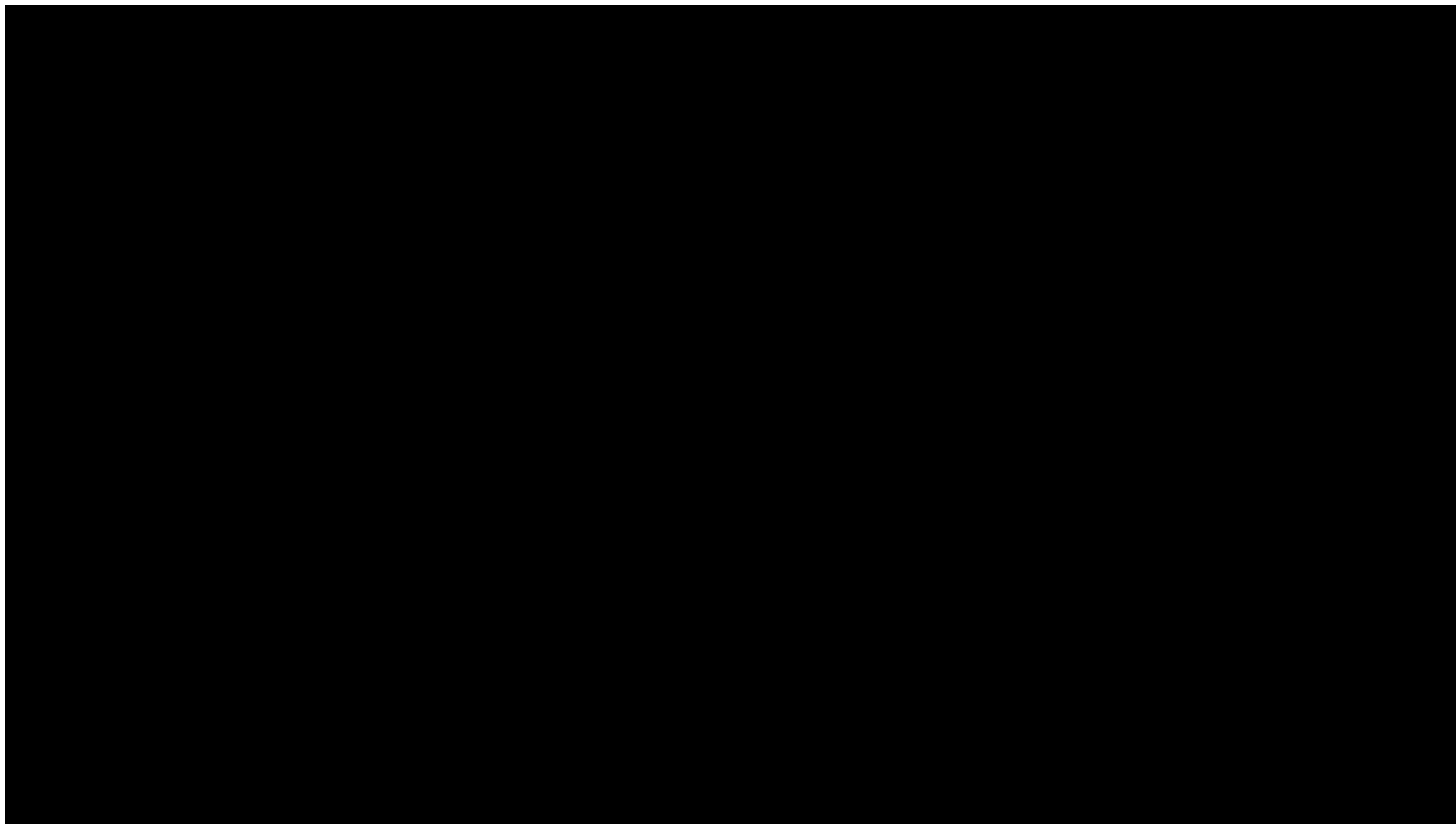


[[TRPO](#)]

[Schulman et al. (2017): Trust Region Policy Optimization]

[Schulman et al. (2017): Proximal Policy Optimization]

Kian Katanforoosh



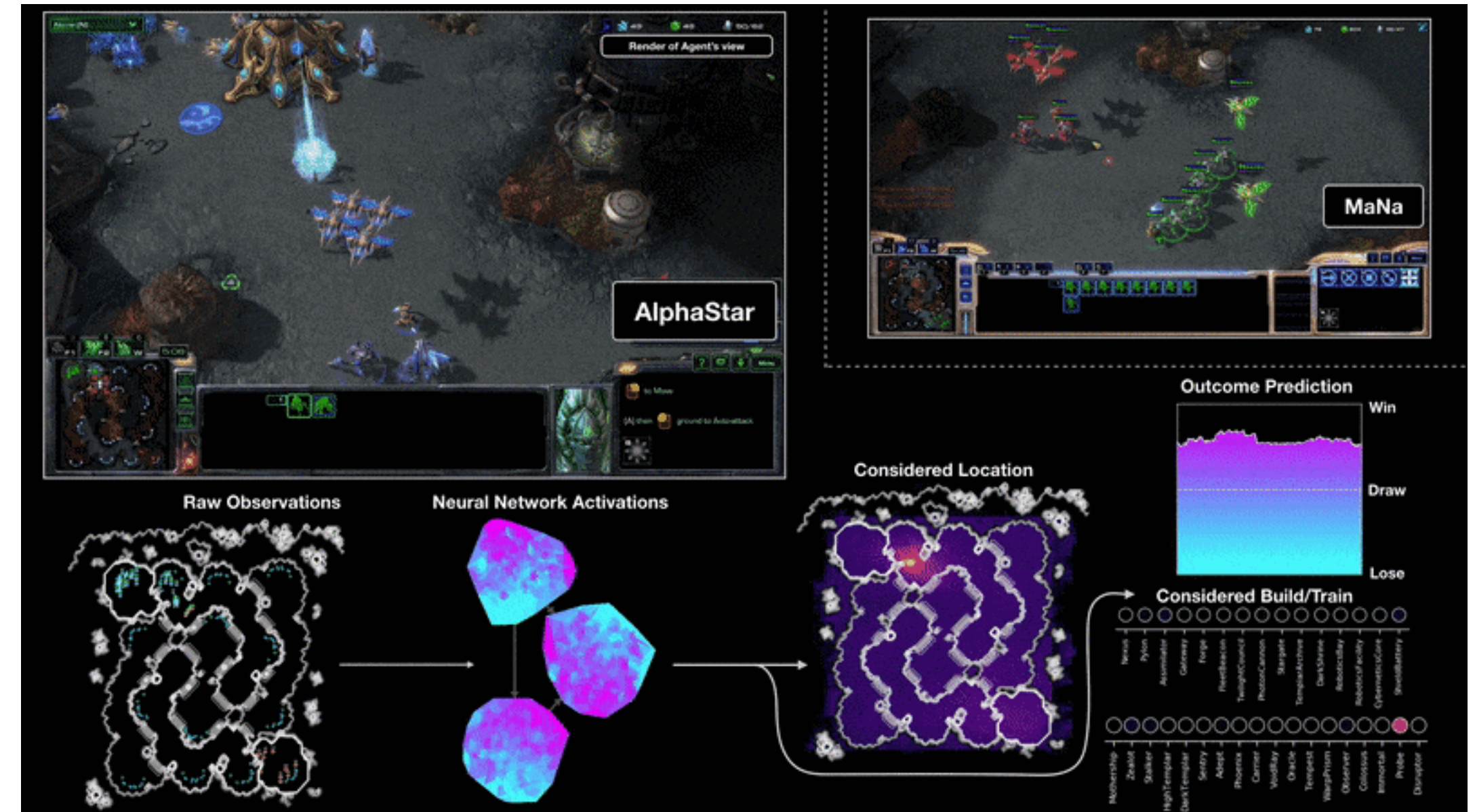
VI - Advanced topics

Open AI Five



[OpenAI Blog Five]

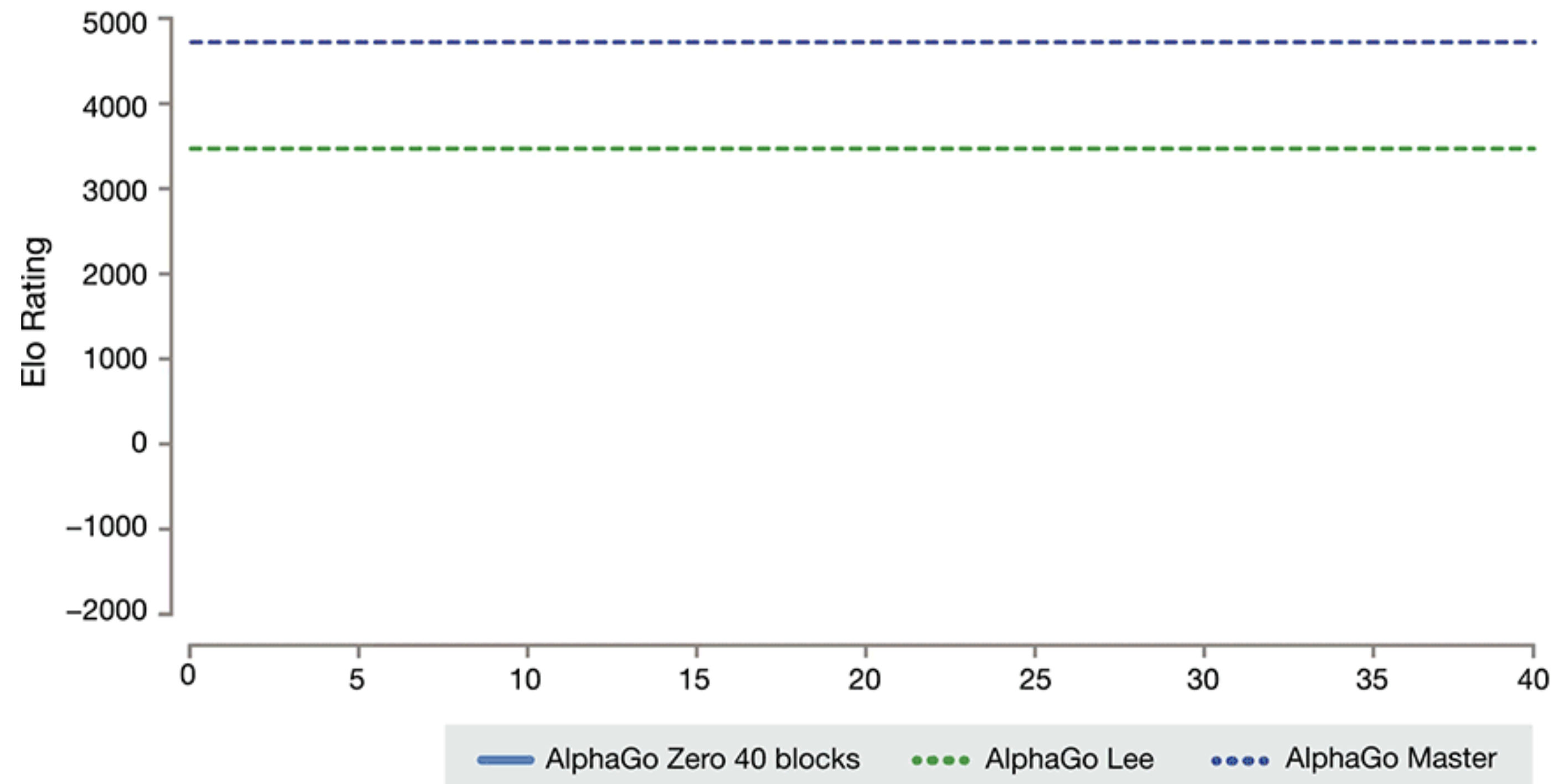
Deep Mind: Alpha Star



AlphaStar: Mastering the Real-Time Strategy Game StarCraft

VI - Advanced topics

Alpha Go



Announcements

Completed modules:

- C5M2: Natural Language Processing and Word Embeddings ([slides](#))
- C5M3: Sequence-to-Sequence Models ([slides](#))

Quizzes (due by 11:00 a.m. PST, 30 minutes prior to the start of lecture time, unless otherwise noted):

- Natural Language Processing and Word Embeddings
- Sequence Models and Attention Mechanism

Programming Assignments (due by 11:00 a.m. PST, 30 minutes prior to the start of lecture time, unless otherwise noted):

- Operations on Word Vectors - Debiasing
- Emojify!
- Neural Machine Translation with Attention
- Trigger Word Detection